



TM671

Erstellen leistungsfähiger mapp View Visualisierungen

Inhaltsverzeichnis

1 Einleitung.....	4
1.1 Lernziele.....	4
1.2 Sicherheitshinweise und Symbole.....	4
2 Dialoge und MessageBoxen.....	6
2.1 Dialoge Überblick.....	6
2.2 Dialog erstellen, öffnen und schließen.....	6
2.3 MessageBox Überblick.....	12
2.4 MessageBox öffnen und Bestätigung auswerten.....	13
3 Rollen und Rechte an Widgets anwenden.....	17
3.1 Rollenbasierte Sichtbarkeit von Widgets.....	18
3.2 Rollenbasierte Bedienung von Widgets.....	20
4 Dynamisieren in der Visualisierung.....	21
4.1 Einfache Sichtbarkeitssteuerung von Widgets.....	21
4.2 Hintergrundbild einer Page.....	25
4.3 Relative und absolute Positionierung innerhalb Container-Widgets.....	27
4.4 Visualisierung auf Größe des Browserfensters skalieren.....	30
5 Variablenarten in mapp View.....	31
5.1 OPC UA Variablen.....	31
5.2 Session Variablen.....	32
5.3 Expressions.....	39
5.4 Snippets.....	45
6 Multi Client Anwendungen.....	48
7 Dynamisch veränderbare Grafiken.....	50
8 Zusammenfassung.....	52

Voraussetzungen

Trainingsmodule	TM611 - Arbeiten mit mapp View
Software	Voraussetzung für die Durchführung dieses Trainingsmoduls ist das im TM611 erstellte mapp View Projekt. Automation Studio 6.0.2 Automation Runtime 6.0 mapp View 6.0.0
Hardware	ARsim

1 Einleitung

Mit den Aufgaben in diesem Trainingsmodul werden die bereits vorhandenen mapp View Grundkenntnisse vertieft. Die Dynamisierung der eigenen Visualisierung, die Verwendung von Dialogen und Meldungsfenstern als erweiterte Aufgaben des Event & Action Systems, sowie die Anpassung der Visualisierung an den Benutzer sind die wesentlichen Schwerpunkte.

1.1 Lernziele

Durch das selbstständige Erarbeiten der bereitgestellten Aufgabenstellungen, werden die bereits erlernten mapp View Basiskenntnisse weiter vertieft und gefestigt.

Nach diesem Trainingsmodul sind Sie in der Lage:

- das Event & Action System für unterschiedlichen Anwendungsfälle zu nutzen.
- das Benutzer-Rollen-System für die Sichtbarkeit und Bedienung von Widgets zu projektieren.
- die Vorgehensweisen und Möglichkeiten für die Dynamisierung von Widgets zu beschreiben.
- die verschiedenen Arten von Variablen in der mapp View Visualisierung richtig einzusetzen.
- den Einsatz von mehreren Visualisierungen zu konfigurieren.
- Visualisierungen für unterschiedliche Endgeräte zu erstellen.
- einen Maschinenprozess optisch darzustellen.

1.2 Sicherheitshinweise und Symbole

Die Sicherheitshinweise werden im vorliegenden Handbuch wie folgt gestaltet:



Gefahr: Bei Missachtung der Sicherheitsvorschriften und -hinweise besteht die Gefahr schwerer Verletzungen, Todesgefahr oder großer Sachschäden.



Warnung: Bei Missachtung der Sicherheitsvorschriften und -hinweise besteht die Gefahr schwerer Verletzungen oder großer Sachschäden.



Vorsicht: Bei Missachtung der Sicherheitsvorschriften und -hinweise besteht die Gefahr von Verletzungen oder von Sachschäden. Wichtige Angaben zur Vermeidung von Fehlfunktionen.

Hinweise und Zusatzinformationen werden im vorliegenden Handbuch wie folgt gestaltet:



Hinweis: Hier werden wichtige Hinweise und Zusatzinformationen bereitgestellt.



Hilfe: Hier wird auf einen Eintrag von Automation Help verwiesen, der weiterführende Informationen, Datenblätter oder Anwenderhandbücher enthält.

Beispiel: [Programmierung \ Variablen und Datentypen \ Datentypen \ Einfache Datentypen](#)

Mit Klick auf den Link wird Automation Help geöffnet.



Beispiel: Hier wird eine beispielhafte Darstellung gezeigt, die das Gelernte vertieft.



Resultat: Hier wird das Ergebnis einer vorangegangenen Aufgabenstellung kurz zusammengefasst.

Gestaltung der Sicherheitshinweise in externen Handbüchern:

In diesem Handbuch wird auf andere Handbücher verwiesen. Die Gestaltung der Sicherheitshinweise ist im jeweiligen externen Handbuch beschrieben.

Aufgabe: Aufgabenstellungen & Übungsaufgaben

In den grau hinterlegten Abschnitten sind Aufgabenstellungen sowie die zugehörigen Handlungsschritte beschrieben. Die Aufgabenstellungen dienen zur Vertiefung der bereitgestellten Informationen.

2 Dialoge und MessageBoxen

Ein **Dialog** ist ein Visualisierungselement, welches über einer Visualisierungsseite eingeblendet wird.

Eine **MessageBox** ist ein Sonderfall eines Dialogs. Im Gegensatz zum normalen Dialog wird der Inhalt nicht projiziert, sondern beim Aufruf der öffnenden Aktion als Parameter übergeben.



[Visualization \ mapp View \ Engineering \ Dialog boxes](#)

2.1 Dialoge Überblick

Ein Dialog wird wie eine Visualisierungsseiten projiziert, d.h. dem Dialog wird ein Layout zugewiesen und den Areas des Layouts werden Contents zugewiesen. Der Content kann beliebig gestaltet werden.

Ein Dialog steht zur Laufzeit innerhalb einer Visualisierung zur Verfügung und kann als Reaktion auf ein Ereignis durch die Aktion "OpenDialog" angezeigt und durch die Aktion "CloseDialog" geschlossen werden.

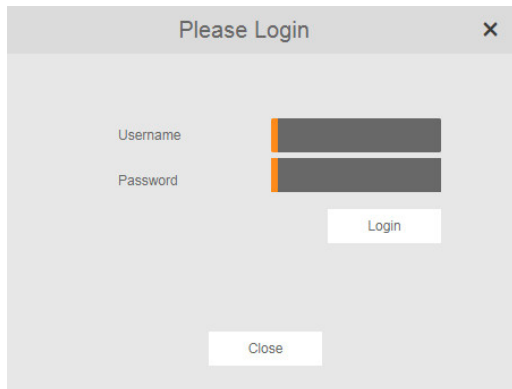


Abbildung 1: Beispiel für einen Dialog

Einsatzgebiete von Dialogen

Dialoge werden eingeblendet, um Eingaben vom Benutzer abzuholen. Modale Dialoge sperren den Rest der Benutzeroberfläche, solange der Dialog angezeigt wird. Nicht-modale (modeless) Dialoge erlauben auch Interaktionen außerhalb des Dialogs.

2.2 Dialog erstellen, öffnen und schließen

Mit Hilfe der nachfolgenden Aufgaben soll ein Dialog erstellt werden, in welchem ein Login-Widget angezeigt wird. Der Dialog wird mit einem Klick auf das Image in der Kopfzeile geöffnet. Bei erfolgreichem Login soll der Dialog automatisch geschlossen werden.

Folgende Schritte sind notwendig:

- 1) [2.2.1 "Layout erstellen" auf Seite 7](#)
- 2) [2.2.2 "Dialog mit Content erstellen" auf Seite 8](#)
- 3) [2.2.3 "EventBinding zum Öffnen des Dialogs" auf Seite 9](#)
- 4) [2.2.4 "EventBinding zum Schließen des Dialogs" auf Seite 11](#)

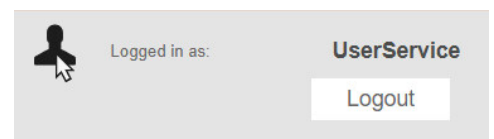


Abbildung 2: Öffnen eines Dialoges durch Klick auf ein Image

2.2.1 Layout erstellen

Aufgabe: Layout für Dialog erstellen

Ziel dieser Aufgabe ist das Erstellen eines Layouts für den Dialog. Das Layout besteht aus einer einzigen Area mit der identen Größe des Layouts.

- 1) In der Logical View unter "mapp View / Visualization" auf "Layouts" klicken
- 2) Neue Layout-Datei einfügen und zu "DialogLayout" umbenennen
- 3) Die Layout-Datei öffnen und die Größe des Layouts ändern:

Common \ Name	Layout \ width	Layout \ height
DialogLayout	450	300

- 4) Die Eigenschaften der Area ändern und Position bzw. Größe anpassen:

Common \ Name	Layout \ top	Layout \ left	Layout \ width	Layout \ height
AreaDialog	0	0	450	300



Ergebnis:

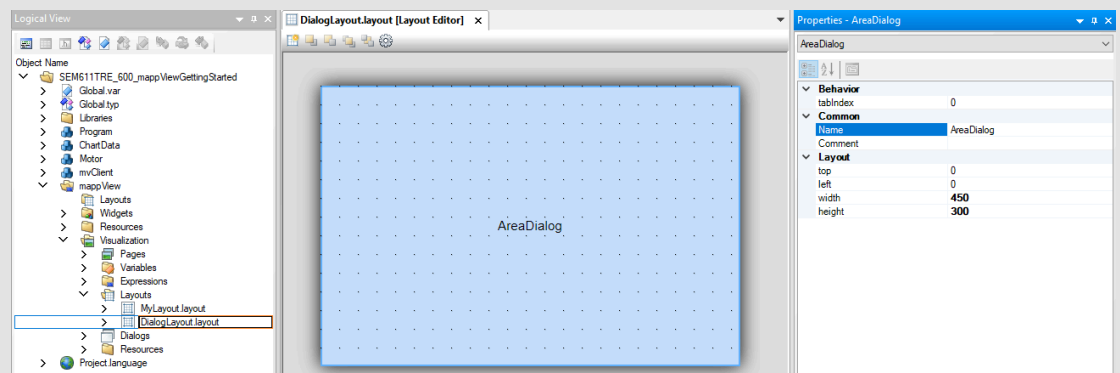
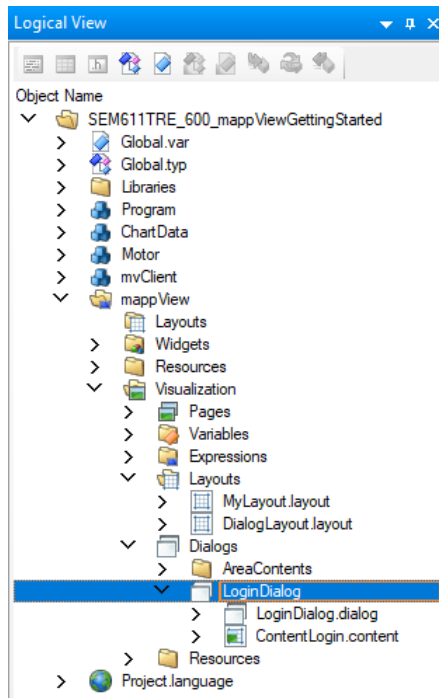


Abbildung 3: Layout Datei in Logical View eingefügt und Eigenschaften bearbeitet

2.2.2 Dialog mit Content erstellen



In mapp View werden Dialoge in der Logical View unter "Visualization \ Dialogs" verwaltet. Ein Dialog wird aus dem Objektkatalog mittels Drag/Drop oder Doppelklick eingefügt.

Um Dialoge im System zu identifizieren, benötigt jeder einzelne eine eindeutige ID, z. B. "LoginDialog".

Dialoge verwenden, wie Visualisierungsseiten, Layouts, z. B. "DialogLayout". Mit Doppelklick auf die Area wird der dazu passende Content erstellt, z. B. "ContentLogin".

Abbildung 4: Eingefügter Dialog mit Content

Aufgabe: Dialog und Content erstellen

- 1) In der Logical View unter "mapp View \ Visualization" auf "Dialogs" klicken
- 2) Neue Dialog-Datei einfügen
- 3) Das Dialog Package zu "LoginDialog" umbenennen und die Datei anschließend öffnen
- 4) Das zuvor erstellte Layout "DialogLayout" referenzieren

Common \ dialogId

LoginDialog

Common \ layoutId

DialogLayout

- 5) Doppelklick auf "AreaDialog" und als Contentnamen "ContentLogin" eingeben. Dadurch wird ein Content mit dem Namen "ContentLogin" erstellt
- 6) "ContentLogin" öffnen
- 7) Login-Widget einfügen und Eigenschaften ändern: "Position" = 80; 80



Ergebnis:

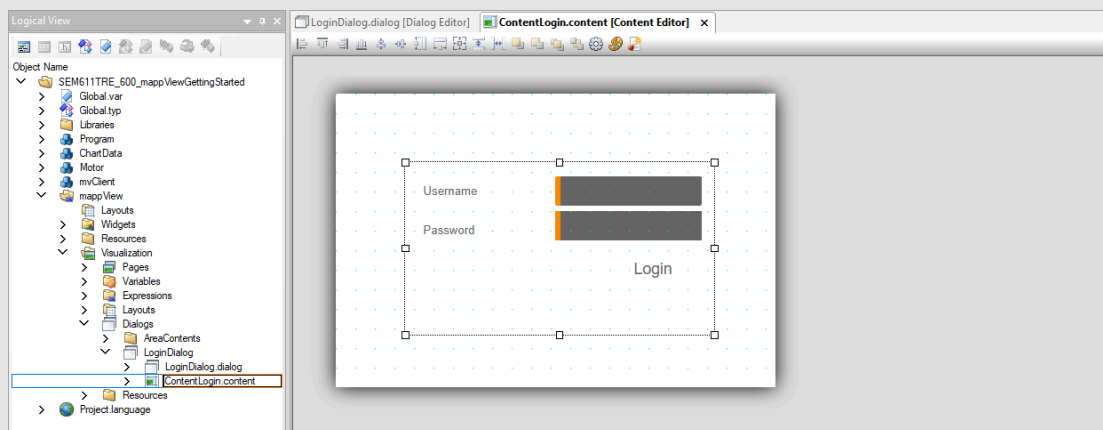


Abbildung 5: Content, welcher im Dialog "LoginDialog" referenziert wird

2.2.3 EventBinding zum Öffnen des Dialogs

Der Dialog soll über einen Klick auf das User Icon, das sich im ContentTop befindet, geöffnet werden. Dafür wird ein ein EventBinding mit "Click"-Ereignis für das Image-Widget "ImageLoginSymbol" erstellt.

Aufgabe: EventBinding für das Öffnen des Dialoges

- 1) "ContentTop" öffnen
- 2) Im Eigenschaftsfenster des Image-Widgets "ImageLoginSymbol" auf "Ereignisse" klicken
- 3) Auf das Feld "Click" klicken und den Auswahldialog öffnen
- 4) Das EventBinding aufklappen
- 5) Aus dem Objektkatalog die "clientSystem.Action" auf das EventBinding ziehen
- 6) Die Aktion markieren und die Eigenschaften ändern:

method	dialogId \ static	headerText \ static
OpenDialog	LoginDialog	Please Login



Ergebnis:

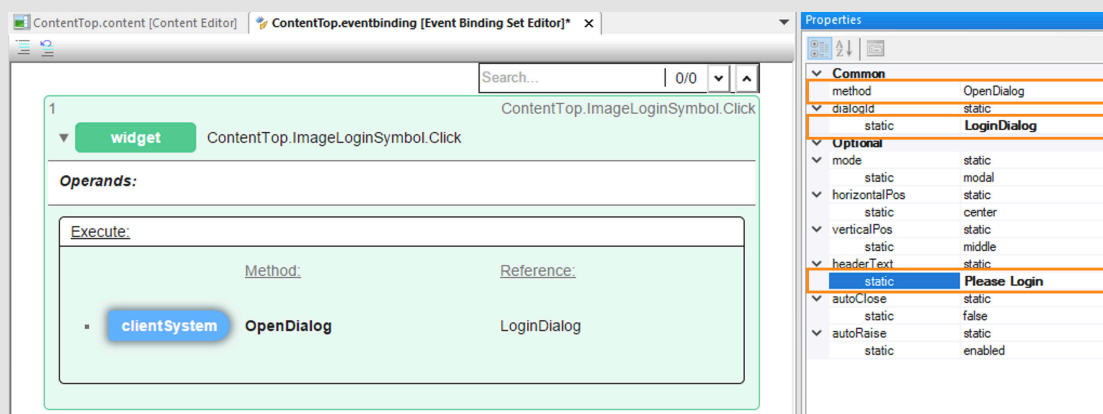


Abbildung 6: Fertiges EventBinding zum Öffnen eines Dialogs

Ergebnis in der Visualisierung

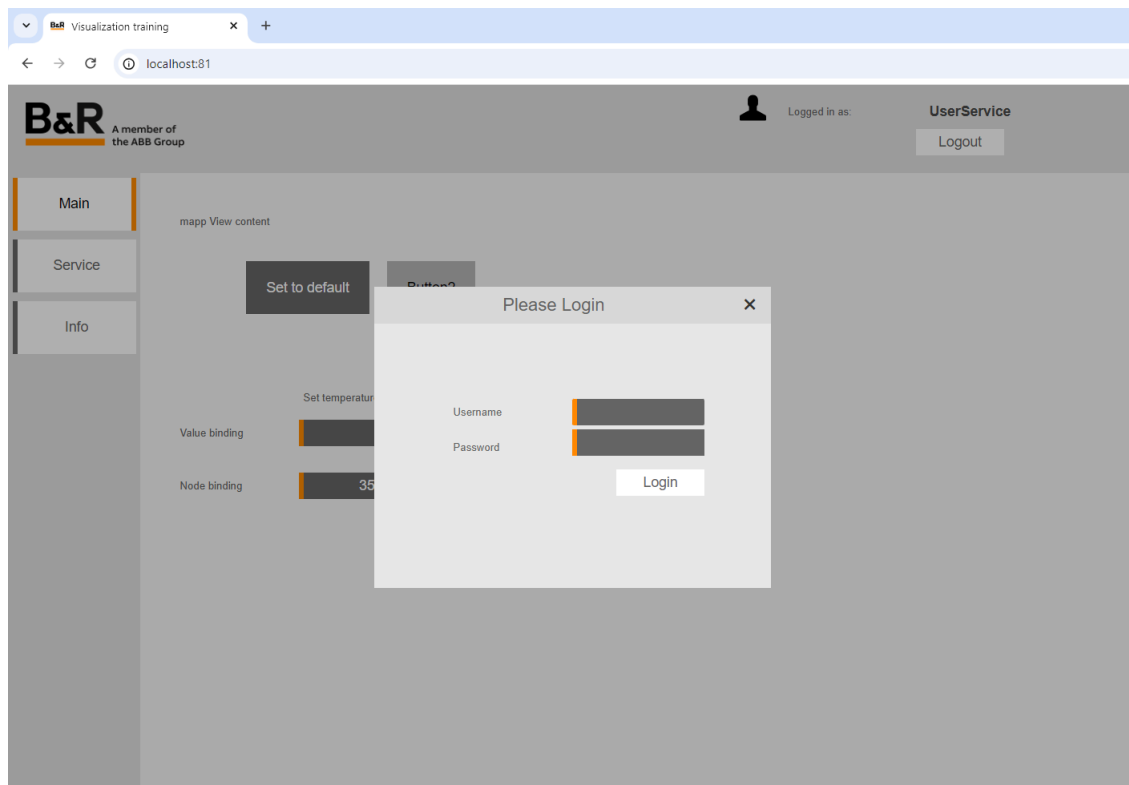


Abbildung 7: Visualisierung im Browser mit geöffnetem Dialog nach Klick auf das User-Image

2.2.4 EventBinding zum Schließen des Dialogs

Das Login-Widget stellt verschiedene Ereignisse zur Verfügung, die weiter verarbeitet werden können z. B. LoginFailed, LoginSuccess, etc.

In diesem Schritt ist geplant den Dialog zu schließen (Aktion), wenn der Login erfolgreich war (Ereignis).



[Visualization \ mapp View \ Engineering \ Events and actions \ Action \ Client actions \ CloseDialog](#)
[Visualization \ mapp View \ Widgets \ Login \ Login \ Actions and events](#)

Aufgabe: EventBinding zum Schließen des Dialogs erstellen

- 1) "ContentLogin" öffnen und Login-Widget auswählen
- 2) Im Eigenschaftfenster auf "Ereignisse" wechseln
- 3) Auf "LoginSuccess" klicken. Es wird ein noch (leeres) EventBinding erstellt
- 4) Aus dem Objektkatalog ein "clientSystem.Action" Element ins EventBinding einfügen
- 5) Eigenschaften der clientSystem.Action anpassen:

method

CloseDialog

dialogId \ static

LoginDialog



Ergebnis:

Das fertige EventBinding sieht folgendermaßen aus:

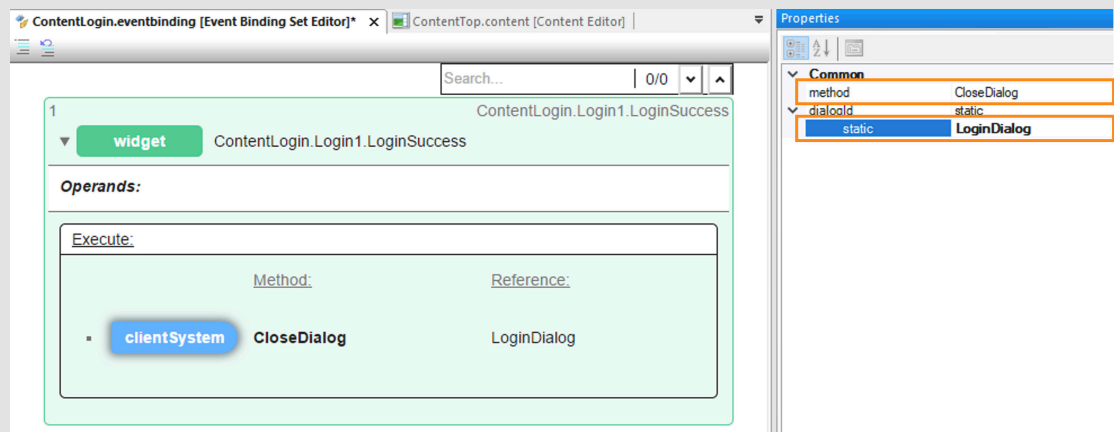


Abbildung 8: EventBinding zum automatischen Schließen des Dialogs bei erfolgreichem Login

Im Browser wird der Login Dialog nach erfolgreichem Login nun automatisch geschlossen.

2.3 MessageBox Überblick

Eine MessageBox wird über einer Page angezeigt. Der Benutzer muss die MessageBox bestätigen, je nach Bestätigung kann eine weitere Aktion ausgeführt werden.

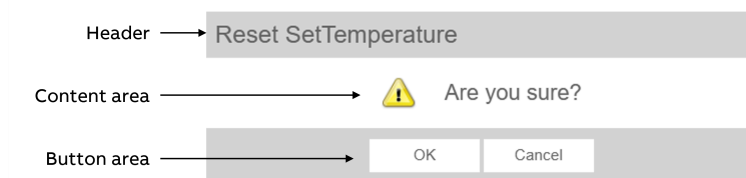


Abbildung 9: Bereiche einer MessageBox

Eine MessageBox besteht aus einem Inhaltsbereich, einer Kopfzeile (=Header) und einer Buttonzeile. Im Inhaltsbereich wird der anzuzeigende Text dargestellt und optional ein Bild. Der Header der MessageBox kann einen Text enthalten. Die Buttonzeile kann einen oder mehrere Buttons enthalten.

Einsatzgebiete von MessageBoxen

MessageBoxen werden eingeblendet, um Eingaben vom Benutzer abzuholen. Eine MessageBox benötigt eine Bestätigung, durch welche der Prozessablauf entsprechend der Auswahl vom Benutzer beeinflusst wird. Im Unterschied zu einem Dialog ist der Aufbau einer MessageBox vom Anwender nicht veränderbar. Eine MessageBox ist eine Systemkomponente.

2.3.1 MessageBox Typen

Durch die Angabe des Typs einer MessageBox wird vorgegeben, welche Buttons beim Aufruf der MessageBox angezeigt werden. Der Typ wird je nach der zu erwartenden Bestätigung ausgewählt, welcher durch das Ereignis erwartet wird.

MessageBox Typ	Beschreibung
AbortRetryIgnore	Zeigt Buttons zum Abbrechen, Wiederholen und Ignorieren
OK	Zeigt einen Button zur Bestätigung
OKCancel	Zeigt Buttons zur Bestätigung und zum Abbruch
RetryCancel	Zeigt Buttons zum Wiederholen und zum Abbrechen
YesNo	Zeigt Buttons für Ja und Nein
YesNoCancel	Zeigt Buttons für Ja, Nein und zum Abbrechen

Tabelle 1: MessageBox Typen

2.3.2 MessageBox Rückgabewert

Wird eine MessageBox von einem bestimmten Typ angezeigt, wird vom Bediener der Visualisierung eine Bestätigung erwartet. Je nach gedrücktem Button kann im EventBinding das Resultat ausgewertet und die gewünschte Aktion projiziert werden.

Jedem Button ist eine ID zugewiesen, durch welcher er in einer Bedingung (condition) ausgewertet werden kann.

Button-ID	Yes	No	Ok	Cancel	Abort	Retry	Ignore
Wert	1	2	4	8	16	32	64

Tabelle 2: MessageBox Button-IDs

2.4 MessageBox öffnen und Bestätigung auswerten

Ziel der nachfolgenden Aufgaben ist es, eine "OKCancel" MessageBox zu projektieren, welche durch ein bestimmtes Ereignis (dem Klick auf einen Button) geöffnet wird. Beim Drücken des "OK" Buttons wird eine OPC UA Variable mit einem definierten Wert beschrieben.

Die MessageBox wird über das EventBinding projiziert.

Folgende Schritte sind notwendig:

- 1) [2.4.1 "Button zum Öffnen der MessageBox konfigurieren" auf Seite 13](#)
- 2) [2.4.2 "EventBinding zum Öffnen der MessageBox konfigurieren" auf Seite 14](#)
- 3) [2.4.3 "Aktionen der MessageBox verwalten" auf Seite 15](#)

2.4.1 Button zum Öffnen der MessageBox konfigurieren

Für das Öffnen der MessageBox wird das bereits im "ContentMainPage" platzierte Button-Widget verwendet.

Aufgabe: Button zum Öffnen der MessageBox konfigurieren

- 1) "ContentMainPage" öffnen
- 2) Den rechten Button zu "ButtonShowMsgBox" umbenennen und angezeigten Text anpassen zu "MsgBox"
- 3) EventBinding für das "Click"-Ereignis des Buttons anlegen



Ergebnis:

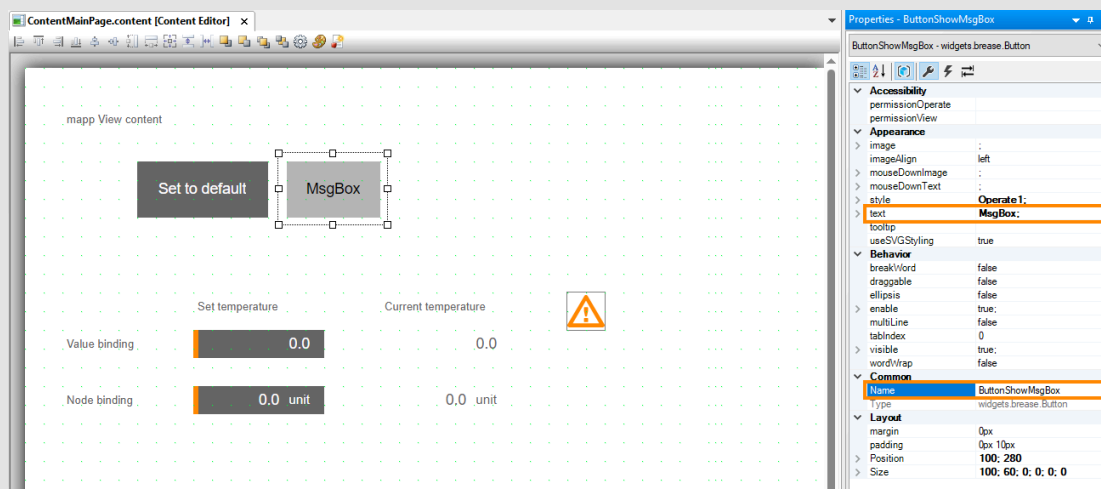


Abbildung 10: Button für das Öffnen einer MessageBox vorbereiten

2.4.2 EventBinding zum Öffnen der MessageBox konfigurieren

Das Ereignis, das die MessageBox öffnen soll, ist ein Klick auf den in der letzten Aufgabe konfigurierten "ButtonShowMsgBox", für den auch bereits ein leeres EventBinding besteht. Nun wird noch eine Aktion angegeben, die auf das Ereignis folgt.



[Visualization \ mapp View \ Engineering \ Events and actions \ Action \ Client actions \ ShowMessageBox](#)

Aufgabe: EventBinding zum Öffnen der MessageBox vervollständigen

- 1) EventBinding Datei öffnen und das EventBinding zum "Click"-Ereignis von "ButtonShowMsgBox" aufklappen
- 2) Aus dem Objektkatalog ein "clientSystem.Action" Element hinzufügen
- 3) Eigenschaften der Aktion verwalten:

method	type \ static	message \ static	header \ static	icon \ static
ShowMessageBox	OKCancel	Are you sure?	Reset SetTempera- ture	Warning



Ergebnis:

Abbildung 11: ClientSystem Aktion "ShowMessageBox"



Der Meldungstext (message) und der Text der Kopfzeile (header) können durch Angabe einer lokalisierten Text-Id aus dem Textsystem referenziert werden (z. B. \$IAT/MessageText).

2.4.3 Aktionen der MessageBox verwalten

Im letzten Schritt wird konfiguriert, welche Aktion ausgeführt werden soll, wenn der Benutzer über die MessageBox ein "OK" absetzt. Wenn "OK" abgesetzt wird, soll die OPC UA Variable "AsGlobalPV:SetTemperature" mit dem Wert 30 beschrieben werden.

Dazu wird im EventBinding, in der Aktion ShowMessageBox, im ResultHandler, eine OPC UA Aktion eingefügt. Da der "OK" Button der MessageBox den Wert "4" zurückgibt (vgl. [2.3.2 "MessageBox Rückgabewert"](#) auf Seite 12) muss in der Bedingung des ResultHandlers "result=4" projiziert werden.

Aufgabe: Aktionen der MessageBox im EventBinding fertig stellen

- 1) In das Feld "Results:" des EventBindings aus dem Objektkatalog ein "opcUa.NodeAction" Element einfügen
- 2) Für "Execute" in den Eigenschaften die "condition" auf "result=4" setzen
- 3) Eigenschaften zur opcUa.nodeAction anpassen:

refId	method	value \ static
()::AsGlobalPV:SetTemperature	SetValueNumber	30



Ergebnis:
Das gesamte EventBinding zum Öffnen und der von der MessageBox bewirkten Aktion sieht folgendermaßen aus:

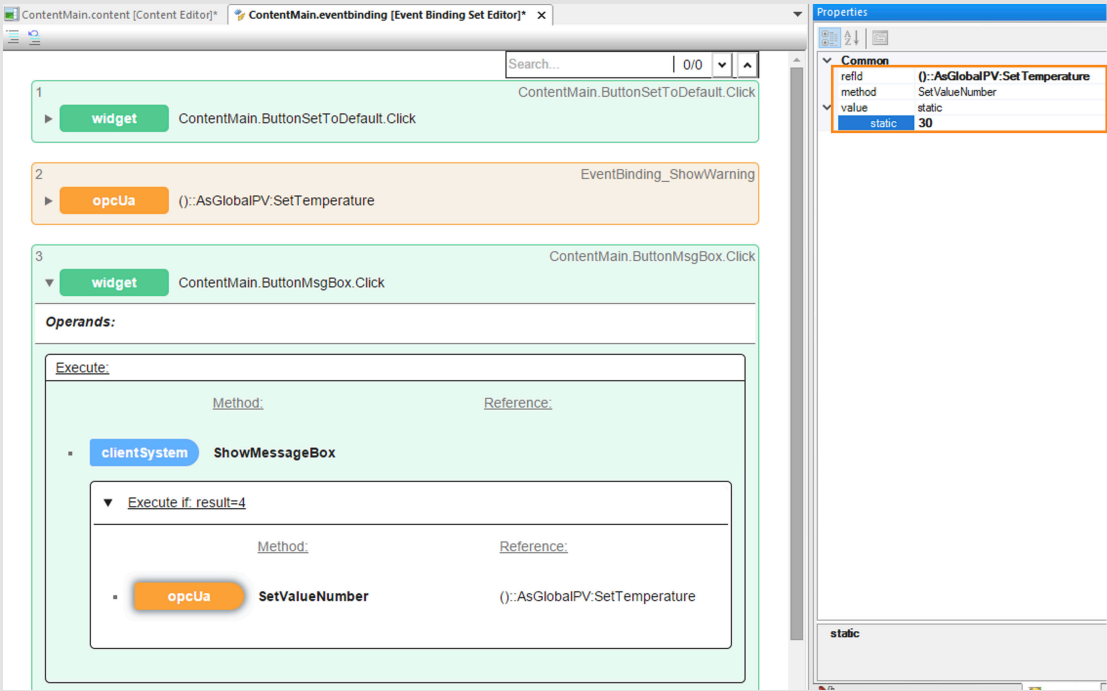
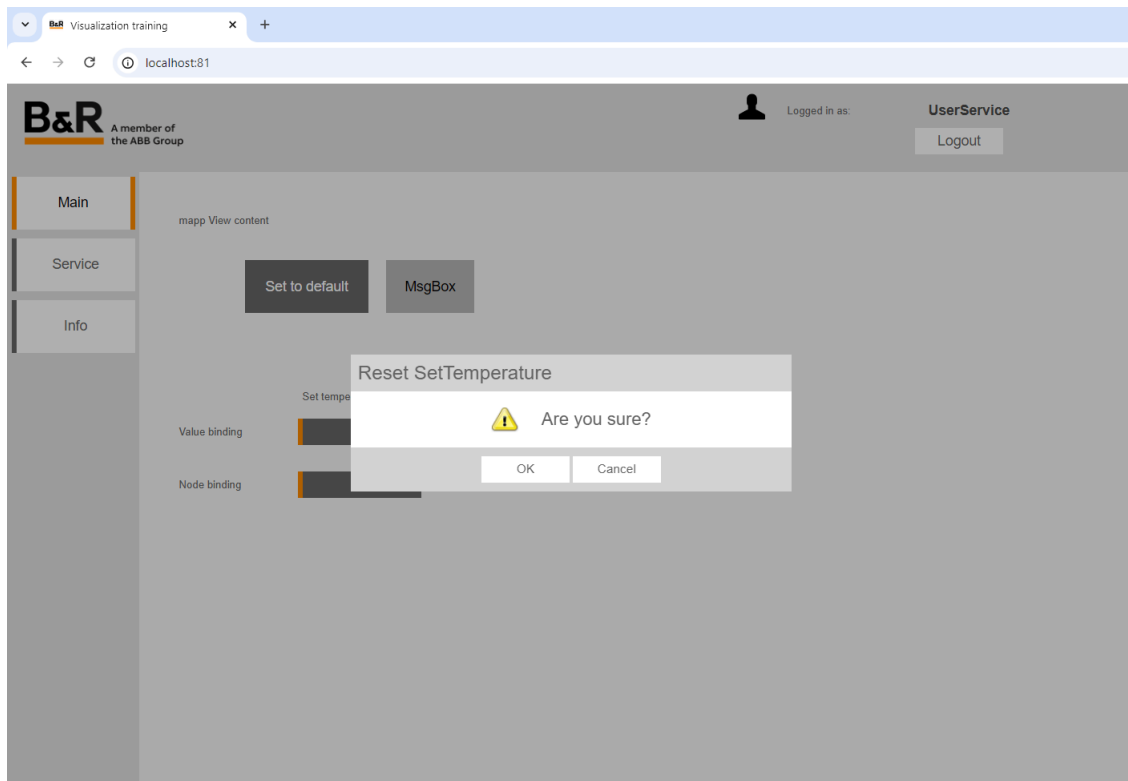


Abbildung 12: Aktion ShowMessageBox mit Aktion in einem ResultHandler

Ergebnis in der Visualisierung

Nach Kompilieren und Übertragen der konfigurierten MessageBox kann diese durch einen Klick auf den konfigurierten Button geöffnet werden.



Führt ein Nutzer, der keine Schreibrechte auf der OPC UA Variable "SetTemperature" hat, die Aktion aus, so wird der Wert nicht auf die konfigurierten "30" gesetzt.

Stattdessen wird im Automation Runtime Logger ein entsprechender Fehler "BadUserAccessDenied" eingetragen:

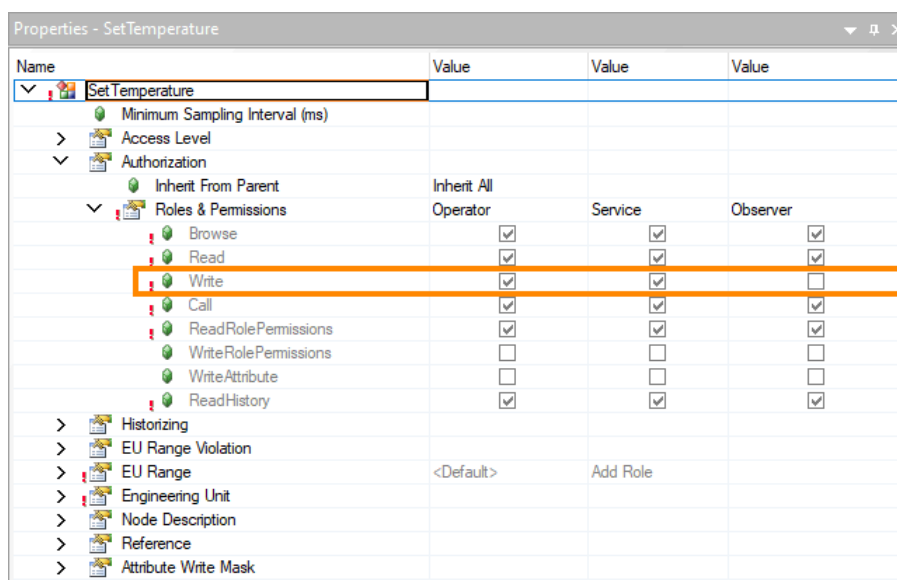
2	Error	-1061091064	mapp View Server	Failed to write OPC UA node attribute (id: NS6(String)::AsGlobalPV:SetTemperature, attribute: Value, serverAlias:). Status: BadUserAccessDenied
---	-------	-------------	------------------	--

3 Rollen und Rechte an Widgets anwenden

Für die Sichtbarkeitssteuerung oder die Bedienung (Enable Zustand eines Widgets) stehen zwei Möglichkeiten zur Verfügung, wobei hier zwischen einer prozessabhängigen- und einer rollenbasierten Funktionalität unterschieden wird. Soll z. B. die Sichtbarkeit oder Bedienbarkeit eines Widgets oder einer Gruppe von Widgets über OPC UA Variablen gesteuert werden (Zustandsabbildung des Prozesses), kann ein Binding an die "visible" bzw. "enable" Eigenschaft eines Widgets durchgeführt werden.

Wie bereits im TM611 gezeigt, kann der Schreibzugriff auf OPC UA Variablen für individuelle Rollen eingeschränkt werden. Dies wirkt sich bei eingabefähigen Widgets auf deren Enable Zustand aus.

Zusätzlich kann die Sichtbarkeit oder Bedienbarkeit eines Widgets aber auch in Abhängigkeit der durch den eingeloggteten Benutzer zugewiesene Rollen eingeschränkt werden.



Name	Value	Value	Value
Set Temperature			
Minimum Sampling Interval (ms)			
Access Level			
Authorization			
Inherit From Parent	Inherit All		
Roles & Permissions	Operator	Service	Observer
Browse	☒	☒	☒
Read	☒	☒	☒
Write	☒	☒	☐
Call	☒	☒	☒
ReadRolePermissions	☒	☒	☒
WriteRolePermissions	☐	☐	☐
WriteAttribute	☐	☐	☐
ReadHistory	☒	☒	☒
Historizing			
EU Range Violation			
EU Range	<Default>	Add Role	
Engineering Unit			
Node Description			
Reference			
Attribute Write Mask			

Abbildung 13: Eingeschränkte Schreibrechte einer OPC UA Variable für die Rolle "Observer"



[Visualization \ mapp View \ Guides \ FAQ \ Widget applications \ Limiting visibility and operation](#)

3.1 Rollenbasierte Sichtbarkeit von Widgets

Die **Sichtbarkeit** eines beliebigen Widget wird über dessen Eigenschaft **"permissionView"** für ausgewählte Rollen konfiguriert.

Aufgabe: Projektieren der Rechte für Sichtbarkeit eines NavigationButton

Ziel dieser Aufgabe ist es, die Sichtbarkeit des NavigationButton zur Navigation auf die ServicePage für die Rolle "Observer" einzuschränken.

- 1) "ContentLeft" öffnen und "Service" Button auswählen
- 2) In den Eigenschaften bei "permissionView" nur die Rollen "Operator" und "Service" zulassen



Ergebnis:

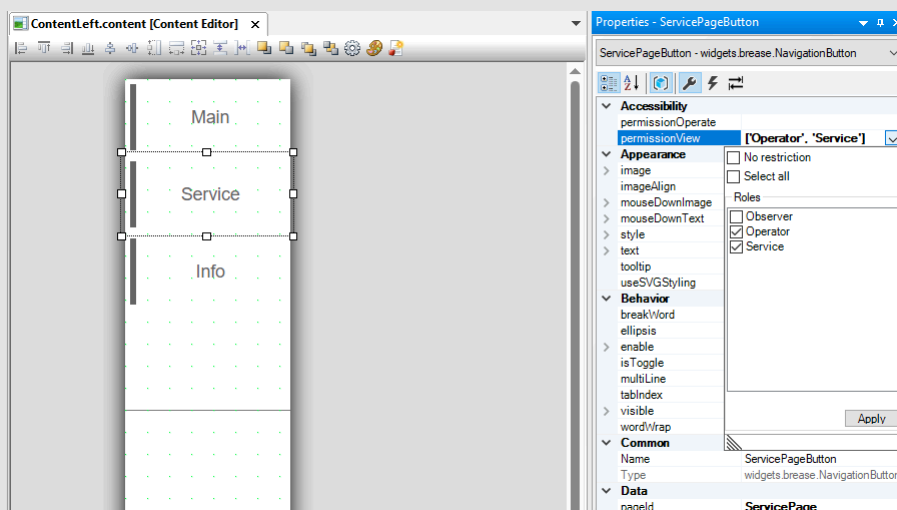


Abbildung 14: User mit der Rolle "Service" oder "Operator" sehen den NavigationButton "Service"

Ergebnis in der Visualisierung

Der Benutzer "UserObserver", der die Rolle "Observer" besitzt, kann den Button zur Navigation auf die ServicePage nun nicht mehr sehen, nur eingeloggten Benutzern mit der Rolle "Operator" oder "Service" ist die Navigation auf die ServicePage möglich.

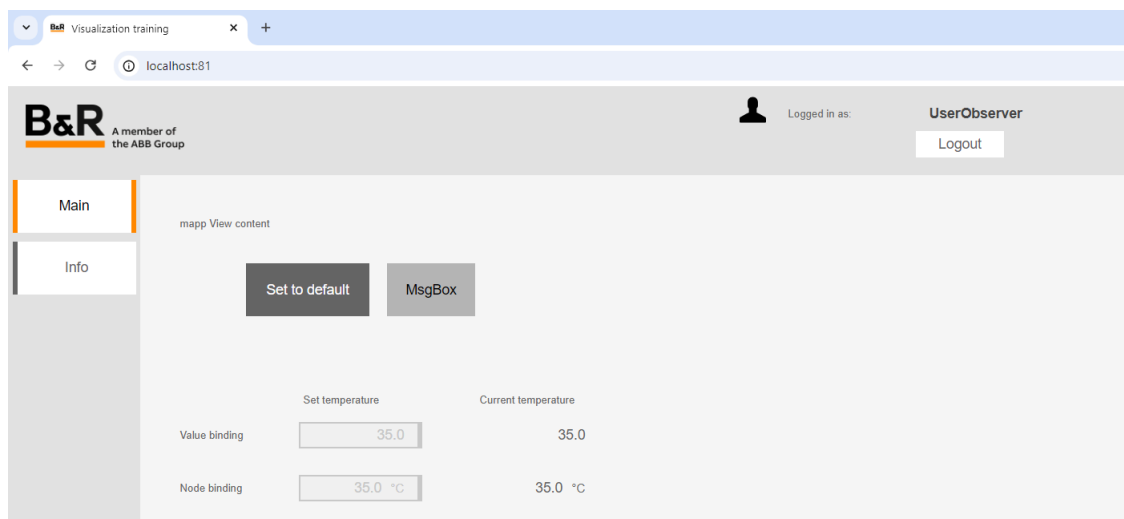


Abbildung 15: NavigationButton "Service" ist für Benutzer "UserObserver" mit der Rolle "Observer" nicht mehr sichtbar



Abbildung 16: NavigationButton "Service" ist für Benutzer "UserService" mit der Rolle "Service" bzw. "UserOperator" mit der Rolle "Operator" sichtbar



Für NavigationButtons der automatischen Navigation kann die Sichtbarkeit nicht eingeschränkt werden.

3.2 Rollenbasierte Bedienung von Widgets

Die Erlaubnis zur **Bedienung** eines beliebigen Widget wird über dessen Eigenschaft **"permissionOperate"** für ausgewählte Rollen konfiguriert.

Aufgabe: Projektieren der Rechte für Bedienbarkeit eines NavigationButton

Ziel dieser Aufgabe ist es, zusätzlich zur Sichtbarkeit eines NavigationButton-Widgets die Bedienbarkeit für die Rolle "Operator" für die Navigation auf die ServicePage einzuschränken.

- 1) "ContentLeft" öffnen und "Service" Button auswählen
- 2) In den Eigenschaften bei "permissionOperate" nur die Rolle "Service" zulassen

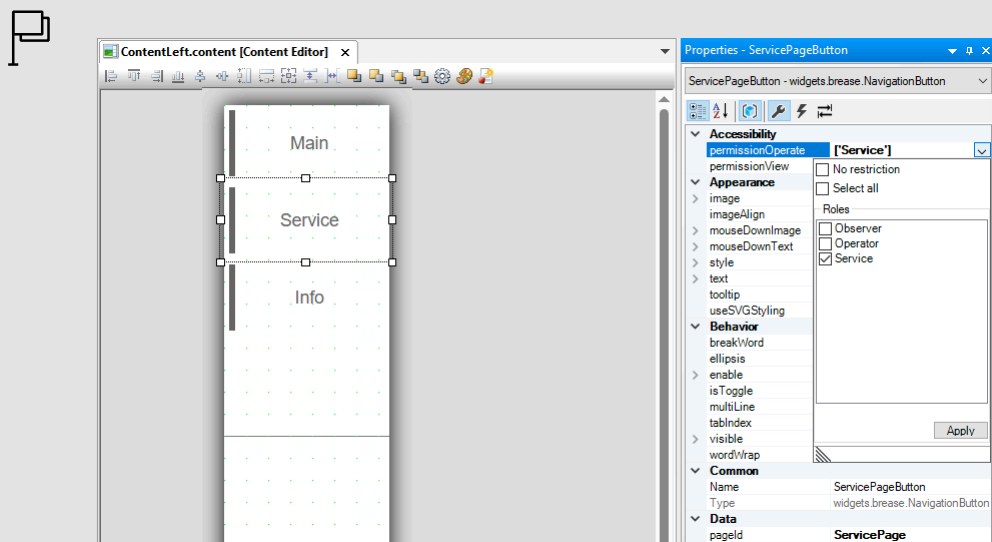


Abbildung 17: User mit der Rolle "Service" können den NavigationButton "Service" bedienen

Ergebnis in der Visualisierung



Abbildung 18: Benutzer der Rolle "Operator" können den NavigationButton sehen, aber nicht bedienen

4 Dynamisieren in der Visualisierung

Mit der Projektierung von statischen Bildanteilen, bei denen die Farbgebung und das Aussehen bereits bei der Projektierung im Content Editor fixiert werden, lassen sich bereits viele Elemente einer ansprechenden Visualisierung erstellen.

Zusätzlich ist es möglich, in Abhängigkeit von Prozesszuständen, die Sichtbarkeit oder Bedienbarkeit eines Widgets oder einer Widget-Gruppe zu verändern bzw. die optische Darstellung eines Widgets zur Laufzeit zu verändern.

Weitere Aufgaben zeigen, wie z. B. eine für eine bestimmte Größe erstellte Visualisierung auf ein Endgerät mit einer anderen Auflösung angepasst werden kann, und wie eine Page mit einem Hintergrundbild projiziert wird.

4.1 Einfache Sichtbarkeitssteuerung von Widgets

Die folgende Aufgabe zeigt, wie auf Basis eines Prozesszustandes, welcher als OPC UA Variable vorliegt, die Sichtbarkeit eines Widgets gesteuert wird.



[Visualization \ mapp View \ Guides \ FAQ \ Widget applications \ Limiting visibility and operation](#)

Folgende Schritte sind notwendig:

- 1) [4.1.1 "ToggleButton-Widget mit Binding erstellen" auf Seite 21](#)
- 2) [4.1.2 "'visible' Eigenschaft eines Widgets konfigurieren" auf Seite 22](#)

4.1.1 ToggleButton-Widget mit Binding erstellen

Um die Änderung eines boolschen Werts zu simulieren, wird im Content der MainPage ein ToggleButton-Widget eingefügt. Der Button soll den Wert der boolschen Variable "State1" umschalten.

Aufgabe: ToggleButton-Widget mit Binding an OPC UA Variable ::Program:State1 erstellen

- 1) Die Variable "::Program:State1" für OPC UA aktivieren
- 2) "ContentMainPage" öffnen
- 3) Ein ToggleButton-Widget einfügen und Eigenschaften konfigurieren:

Appearance \ style	Appearance \ text	Common \ Name
Operate	Visible	ToggleButtonVisibility

- 4) value Binding auf die OPC UA Variable "::Program:State1" ausführen



Ergebnis:

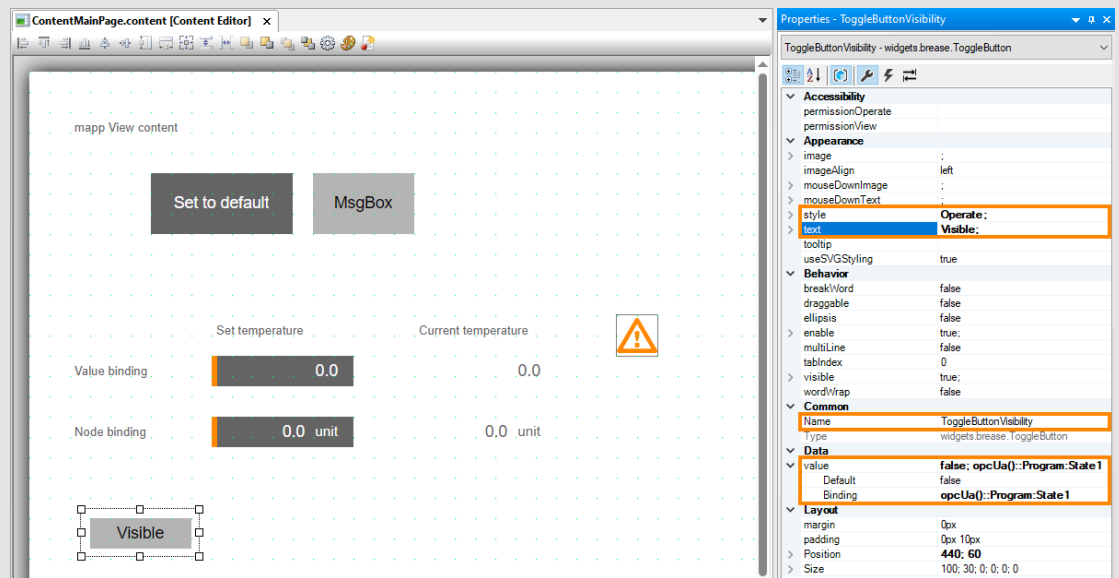


Abbildung 19: Eigenschaften des ToggleButtons



Text und Hintergrundfarbe eines ToggleButton-Widget können für den "gedrückten" und "nicht gedrückten" Zustand gesondert konfiguriert werden.

4.1.2 "visible" Eigenschaft eines Widgets konfigurieren

An der visible Eigenschaft eines NumericOutput-Widgets ist ein Binding an die OPC UA Variable "::Program:State1" durchzuführen.

Um ein Flackern der Sichtbarkeit eines Widgets bei der initialen Darstellung zu vermeiden, muss der Default Wert auf "false" gestellt werden.

Aufgabe: Binding an die "visible" Eigenschaft eines NumericOutput-Widgets

- 1) "ContentMainPage" öffnen
- 2) NumericOutput-Widget "NumericOutputCurrentTempNode" auswählen
- 3) Eigenschaften anpassen:

visible \ Default

false

visible \ Binding

::Program:State1

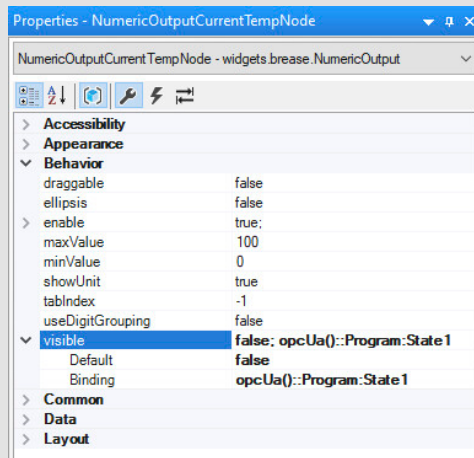
**Ergebnis:**

Abbildung 20: Visible Binding einer OPC UA Variable

Ergebnis in der Visualisierung

Der ToggleButton "simuliert" eine Steuerung der Sichtbarkeit des NumericOutput-Widgets.

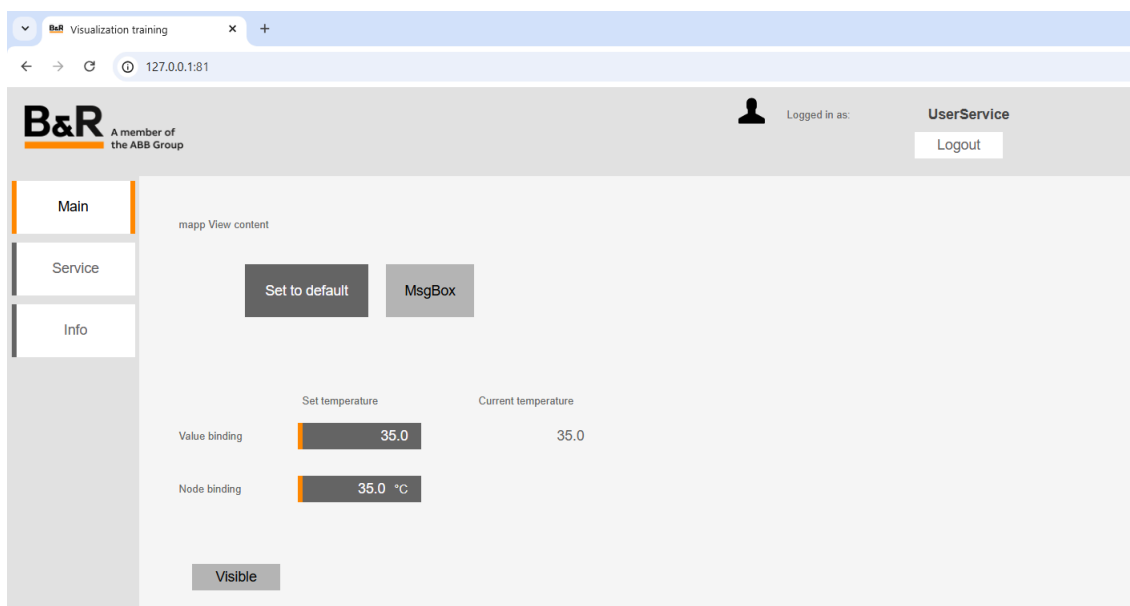


Abbildung 21: Sichtbarkeit des Widgets bei losgelassenem ToggleButton

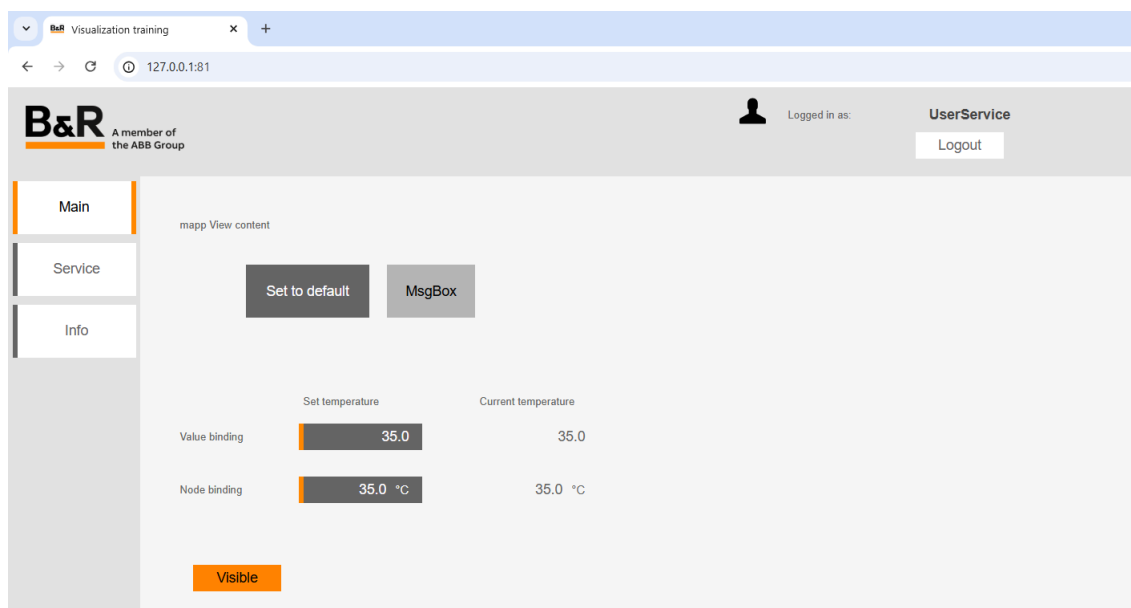


Abbildung 22: Sichtbarkeit des Widgets bei gedrücktem ToggleButton

4.2 Hintergrundbild einer Page

In den bisherigen Aufgaben wurde bei der Zuweisung eines Contents an eine Page auch die Hintergrundfarbe der entsprechenden Area definiert.

Ziel der nachfolgenden Aufgaben ist es, anstelle der Area-Farben ein Image **über die gesamte Page** darzustellen.

Folgende Schritte sind notwendig:

- 1) [4.2.1 "Hintergrundbild zur Verfügung stellen" auf Seite 25](#)
- 2) [4.2.2 "Hintergrundbild in der Page referenzieren" auf Seite 25](#)
- 3) [4.2.3 "Hintergrundfarbe der Areas auf transparent setzen" auf Seite 26](#)

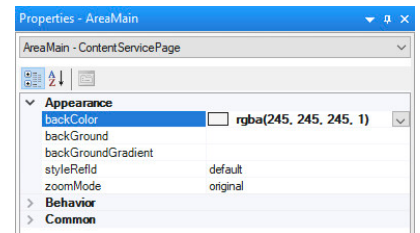


Abbildung 23: Hintergrundfarbe einer Area im Page Editor festlegen

4.2.1 Hintergrundbild zur Verfügung stellen

Falls im Projekt noch kein Hintergrundbild vorhanden ist, muss ein Bild im entsprechenden Seitenverhältnis (im Beispiel 8:5) im MediaPackage der Visualisierung abgelegt werden.



Das Beispielprojekt enthält die Datei "BackGround.png", die hierfür verwendet werden kann.

4.2.2 Hintergrundbild in der Page referenzieren

Nach dem Einfügen in das MediaPackage kann das Bild an einer beliebigen Page referenziert werden. Die Referenz auf das Hintergrundbild wird an der Eigenschaft "backGround" angegeben.

Aufgabe: Hintergrundbild auf ServicePage referenzieren

- 1) "ServicePage" öffnen
- 2) In den Eigenschaften **der Seite, nicht einer einzelnen Area**, bei "backGround" das gewünschte Hintergrundbild referenzieren



Ergebnis:

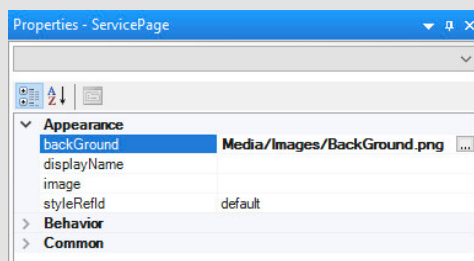


Abbildung 24: BackGround.png ist auf Hintergrund der ServicePage verknüpft

4.2.3 Hintergrundfarbe der Areas auf transparent setzen

Wird das Projekt übertragen zeigt sich noch keine Auswirkung des angehängten Hintergrundbildes. Erst durch das Setzen einer transparenten Hintergrundfarbe aller Areas wird das Hintergrundbild zur Laufzeit sichtbar.

Aufgabe: Alle Areas der ServicePage transparent setzen

Für alle drei Areas der ServicePage sind die folgenden Schritte auszuführen:

- 1) Eigenschaft "backColor" der Area mit einem Rechtsklick und "Reset" zurücksetzen
- 2) Eigenschaft "styleRefId" auf "transparent" setzen



Ergebnis:

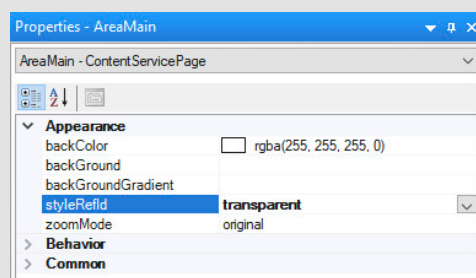


Abbildung 25: Style "transparent" auf alle drei Areas anwenden

Ergebnis in der Visualisierung

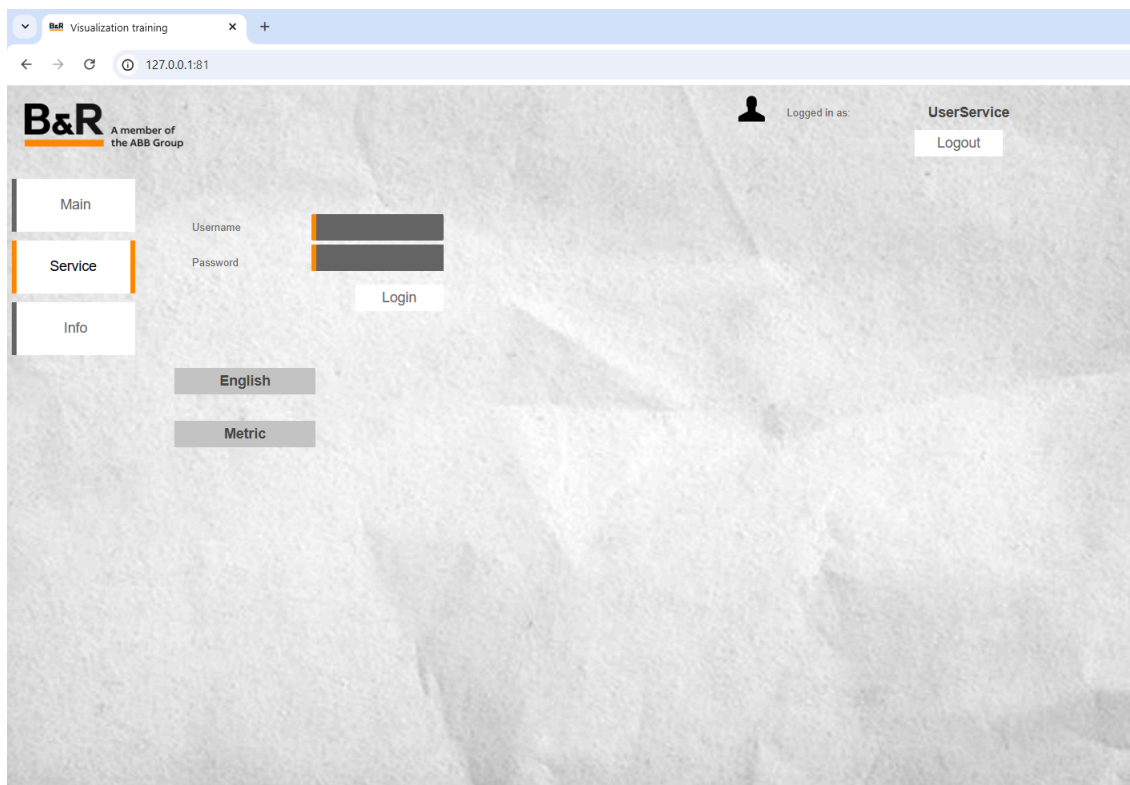


Abbildung 26: Hintergrundbild wird vollflächig auf ServicePage angezeigt

4.3 Relative und absolute Positionierung innerhalb Container-Widgets

In der Navigation in "ContentLeft" wurde bereits die relative Positionierung der NavigationButton-Widgets angewandt. In diesem Kapitel folgen weitere Aufgaben dazu.

Standardmäßig werden Widgets auf dem Content mit einer absoluten top/left Position gezeichnet. Innerhalb von Container-Widgets gibt es die Eigenschaft "**childPositioning**" mit den Auswahlmöglichkeiten "relativ" und "absolut".

- **relativ:** Die Child-Widgets werden automatisch der Reihe nach positioniert. Die Position des eingefügten Child-Widgets muss dabei 0, 0 sein.
- **absolut:** Die Child-Widgets können beliebig positioniert werden.



[Visualization \ mapp View \ Widgets \ Container](#)



Größere Verschachtelungen von Container Widgets wirken sich negativ auf die Performance der Visualisierung aus.

Ziel der folgenden Aufgaben ist es, in einer GroupBox beliebige Widgets zeilenweise zu positionieren.

Jede Zeile ist eine eigene transparente GroupBox, die relativ im Container positioniert ist.

Die Zeilen werden durch ein Binding an der Eigenschaft "visible" einzeln ein- und ausgeblendet.

Folgende Schritte sind notwendig:

- 1) [4.3.1 "Verschachtelte GroupBox erstellen" auf Seite 27](#)
- 2) [4.3.2 "Verschachtelte Child GroupBoxen ein- und ausblenden" auf Seite 29](#)

4.3.1 Verschachtelte GroupBox erstellen

Im ersten Schritt wird auf der rechten Seite der MainPage eine GroupBox eingefügt, die ihrerseits wiederum GroupBox Widgets enthält. Die äußere GroupBox wird dabei als "Parent" bezeichnet, die inneren GroupBoxen als "Childs".

Aufgabe: Verschachtelte GroupBox erstellen

- 1) "ContentMainPage" öffnen und ein GroupBox-Widget einfügen

- 2) Eigenschaften des GroupBox-Widgets konfigurieren:

Name	childPositioning	Position	Size	text
GroupBoxParent	relative	60; 720	380; 360	Parent GroupBox

- 3) Aus dem Objektkatalog drei weitere GroupBox-Widgets in das "GroupBoxParent" einfügen

- 4) Eigenschaften der Child GroupBox-Widgets konfigurieren:

Common \ Name	Appearance \ style	Appearance \ text	Layout \ Position	Layout \ Size
GroupBoxChild1	transparent		0; 0	100%; 90
GroupBoxChild2	transparent		0; 0	100%; 90
GroupBoxChild3	transparent		0; 0	100%; 90

- 5) Je GroupBoxChild ein Label-Widget und ein Button-Widget einfügen

Common \ Name	Appearance \ text	Layout \ Position
LabelChild1	Label Child 1	30; 40
LabelChild2	Label Child 2	30; 40

Common \ Name	Appearance \ text	Layout \ Position
LabelChild3	Label Child 3	30; 40



Durch das Entfernen des Texts einer GroupBox kann diese als "unsichtbarer" Container verwendet werden.

Bei relativ positionierten Child-Widgets darf nur die Breite in "%" angegeben werden. Da die Anzahl der sichtbaren Widgets zur Laufzeit initial nicht bekannt ist, muss die Höhe immer in Pixel angegeben werden.

Ergebnis in der Visualisierung

Auf der rechten Seite der MainPage wird die verschachtelte GroupBox angezeigt. Durch das relative ChildPositioning musste für die inneren GroupBoxen keine explizite Angabe der Position in absoluten Werten erfolgen.

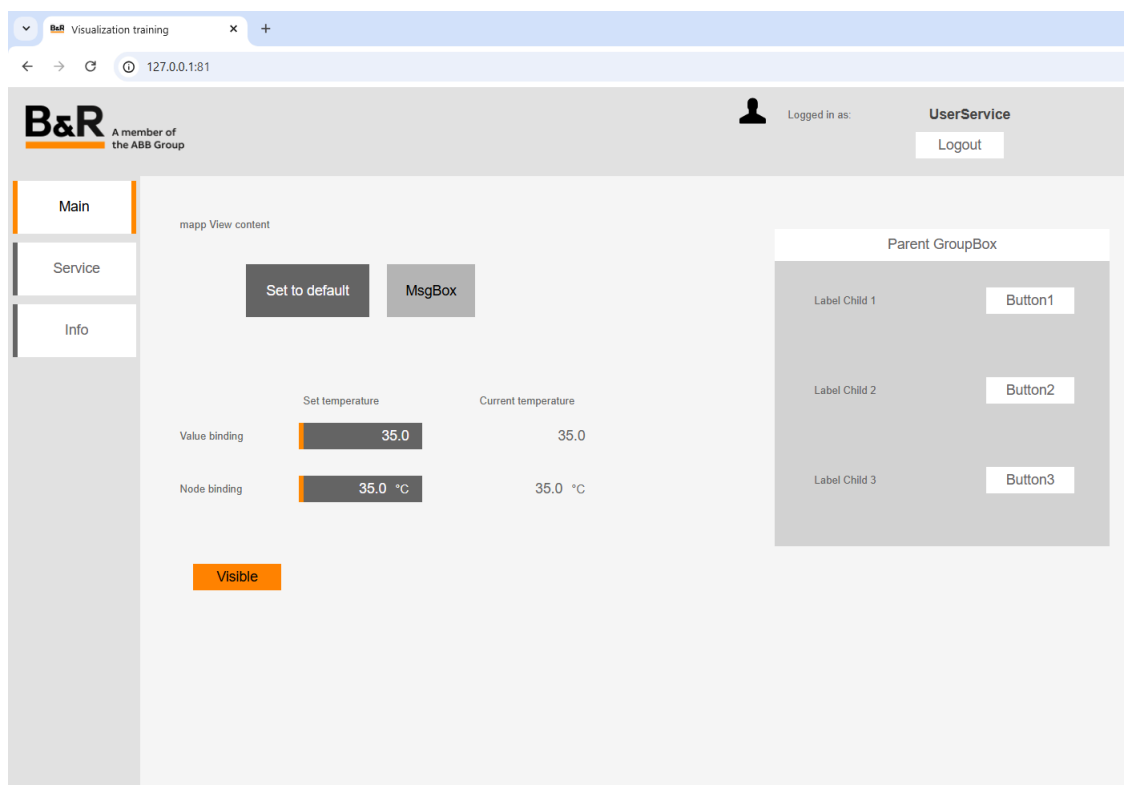


Abbildung 27: Anzeige der verschachtelten GroupBoxen auf der rechten Seite der MainPage

4.3.2 Verschachtelte Child GroupBoxen ein- und ausblenden

Mit der aktuellen Projektierung der verschachtelten GroupBox sind alle drei Zeilen dauerhaft sichtbar. Um einzelne Zeilen, also die child GroupBoxen, in der Parent GroupBox sichtbar oder unsichtbar zu schalten, soll nun bei der ersten und zweiten child GroupBox ein Binding an deren "visible" Eigenschaft erstellt werden.

Aufgabe: "visible" Eigenschaft der Child GroupBoxen konfigurieren

- 1) Die OPC UA Variable "::Program:State2" für das OPC UA System aktivieren (falls noch nicht geschehen)
- 2) Bei den child GroupBoxen "GroupBoxChild1" und "GroupBoxChild2" in den Eigenschaften bei "visible \ Binding" die jeweilige OPC UA Variable ("State1" bzw. "State2") referenzieren
- 3) Zwei ToggleButton-Widgets einfügen

Common \ Name	Appearance \ style	Appearance \ text	Data \ value	Layout \ Position
ToggleButtonVisible1	Operate	Visible 1	State1	60;600
ToggleButtonVisible2	Operate	Visible 2	State2	100; 600

Ergebnis in der Visualisierung

Mit den beiden ToggleButtons, die den Wert der OPC UA Variablen "State1" und "State2" umschalten, können die ersten beiden Zeilen der verschachtelten GroupBox nun sichtbar oder unsichtbar geschaltet werden.

Durch die relative Positionierung der Child-Widgets (Eigenschaft des Parent-Widgets) werden die Child-Widgets frei im Dokumentenfluss eingefügt. Das bedeutet, dass Zeile2 und Zeile3 bei unsichtbarschalten von Zeile1 im Container automatisch nach oben rutscht.

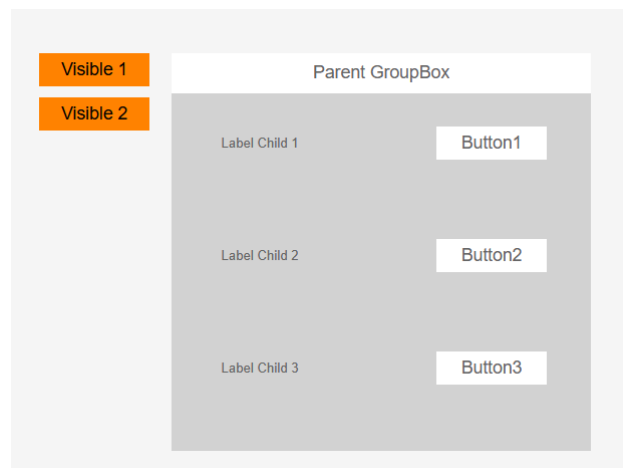


Abbildung 28: beide Zeilen sichtbar

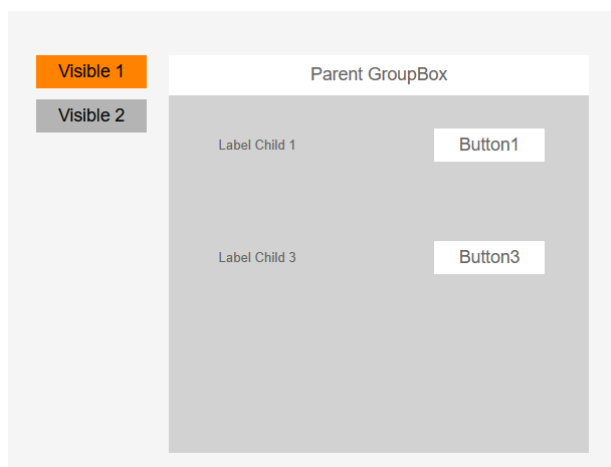


Abbildung 29: Zeile2 unsichtbar geschaltet, Zeile 3 rutscht automatisch nach oben

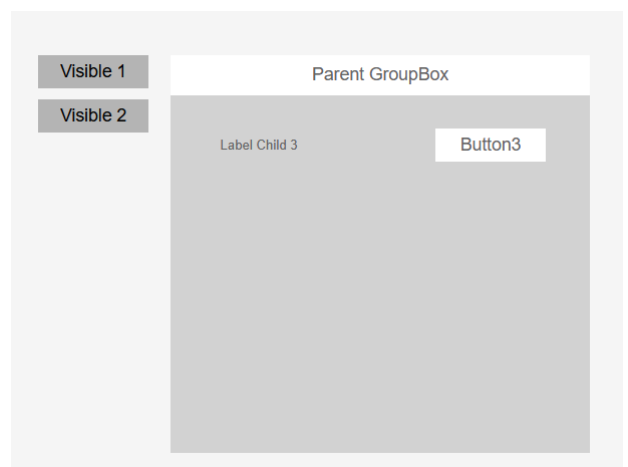


Abbildung 30: Zeile1 und Zeile3 unsichtbar geschaltet, Zeile3 rutscht automatisch nach oben

4.4 Visualisierung auf Größe des Browserfensters skalieren

Eine Visualisierung wird für die Auflösung eines Displays erstellt. Die Größe der Page wird über das Layout festgelegt, welches in einer Page referenziert wird. Besteht die Notwendigkeit, dass die gleiche Visualisierung auf unterschiedlich großen Displays angezeigt werden muss (z. B. HD Ready und Full HD), kann dies durch die Konfiguration eines automatischen "Zoom" konfiguriert werden.



Es ist darauf zu achten, dass die Bedienbarkeit der Visualisierung auf beiden Endgeräten mit unterschiedlicher Diagonale gegeben ist. Des Weiteren muss das Seitenverhältnis der Endgeräten identisch sein (z.B. 4:3 oder 16:9).



[Visualization \ mapp View \ Engineering \ Organization of the HMI application \ mapp View visualization object \ Configurations](#)



Die Zoom Einstellung wird beim Ausliefern der Visualisierung entsprechend der Größe des Browserfensters einmalig angepasst. Ein dynamisches Verändern der Größe zeichnet zwar die Page neu, einige Widgets können aber auf die Größenänderung nicht reagieren und behalten deren Initial-Größe bzw. Eingabefläche.

Aufgabe: Automatische Anpassung der Visualisierung an das Browserfenster

Die Konfiguration der Zoom Einstellung erfolgt in der Visualisierung im Element <Configurations>. Durch den Konfigurationseintrag key="zoom" mit value="true" wird der automatische Zoom aktiviert.

- 1) Das .vis-file in der Configuration View öffnen
- 2) Unter <Configurations> folgenden Zeile für die "zoom" Eigenschaft konfigurieren

```
<Configuration key="zoom" value="true" />
```



Ergebnis:

Eine Größenänderung des Browserfensters und ein Neustart der Visualisierung skaliert die Visualisierung entsprechend mit. Es wird der Inhalt so skaliert, dass er bei maximaler Größe vollständig sichtbar ist.

5 Variablenarten in mapp View

In mapp View stehen verschiedene Arten von Variablen für die Projektierung einer Visualisierung zur Verfügung, die im folgenden kurz vorgestellt werden:

Variablenart	Scope / Gültigkeitsbereich	Beschreibung
OPC UA Variablen	Steuerungsprogramm	Variablen, die im Steuerungsprogramm existieren und "außerhalb" der Visualisierung existieren
Session Variablen	Visualisierung	vom Anwender deklarierte "lokale" Variablen der Visualisierung zum (contentübergreifenden) Datenaustausch innerhalb eines Clients
System Variablen	Visualisierung	von mapp View vordefinierte Session Variablen (z.B. clientInfo.currentPageId)
Expressions	Visualisierung	vom Anwender deklarierte Ausdrücke zur logischen Auswertung von Informationen
Snippets	Visualisierung	formatierte Datenquelle zur Anzeige dynamischer Inhalte in Texten



[Visualization \ mapp View \ Engineering \ Variables and data](#)

5.1 OPC UA Variablen

In den bisherigen Aufgaben wurden bereits OPC UA Variablen verwendet und über ein Node bzw. ein Value Binding an unterschiedliche Eigenschaften der projizierten Widgtes zugewiesen.

In EventBindings können Ereignisse von OPC UA Variablen (z.B. "ValueChanged") verwendet oder Aktionen für OPC UA Variablen (z.B. "SetValue") projiziert werden.



[Visualization \ mapp View \ Engineering \ Variables and data \ OPC UA variables](#)

5.2 Session Variablen

Eine Session Variable ist eine Variable, deren Geltungsbereich (Scope) in einer Session liegt. D. h. ihre Werte gelten innerhalb einer Client-Server Verbindung. Damit ist es möglich, Visualisierungszustände je Client zu speichern. Session Variablen können nur über ein Werte Binding gelesen und beschrieben werden.



[Visualization \ mapp View \ Engineering \ Variables and data \ Session variables](#)

Brease-Brease Binding

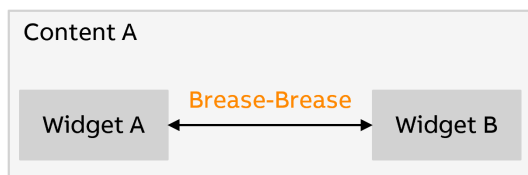


Abbildung 31: Schema Brease-Brease Binding

Befinden sich zwei Widgets im selben Content, so können sie untereinander Informationen über ein einfaches Brease-Brease Binding austauschen. Dabei wird der Wert eines Widgets mit einem Binding direkt an den Wert des anderen Widgets verbunden.

Session Variablen

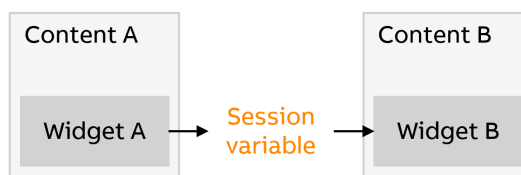


Abbildung 32: Schema Verwendung Session Variable

Befinden sich zwei Widgets in unterschiedlichen Contents, die nicht (oder nicht immer) gleichzeitig auf derselben Seite aktiv sind, müssen Informationen zwischen den Widgets über eine Session Variable ausgetauscht werden.

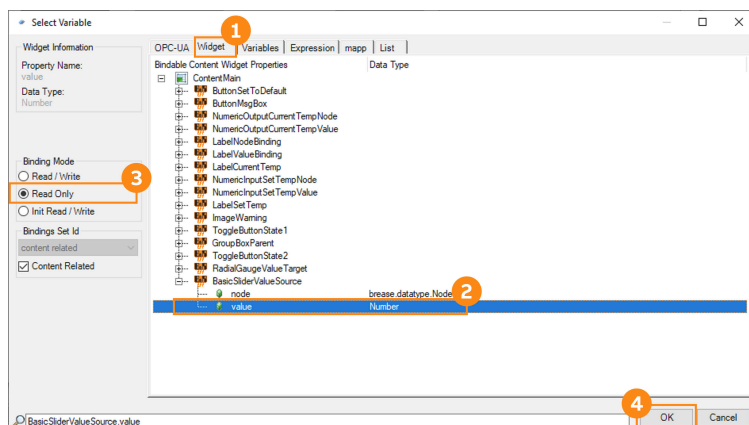


Der Geltungsbereich eines Brease-Brease Bindings ist auf denselben Content beschränkt.

5.2.1 Informationsaustausch zwischen Widgets desselben Contents

Bei einem Brease-Brease Binding gibt es immer eine Source und ein Target (z. B. eine Widgeiteigenschaft). Sowohl Source als auch Target müssen gleichzeitig aktiv sein, damit das Binding funktionieren kann. Wenn der Content der Seite geladen ist, sind die Widgets aktiv.

Ein Brease-Brease Binding zwischen zwei Widgets wird erstellt, indem beim Target Widget in den Eigenschaften für "value" ein Binding erstellt wird:



- 1) Anstatt wie bisher eine OPC UA Variable als Quelle des Bindings zu verwenden, soll ein anderes Widget als Quelle ausgewählt werden
- 2) Es wird auf den Wert des gewünschten Widgets, das die Source des Bindings darstellt, verknüpft
- 3) Binding Mode kann ausgewählt werden
- 4) Binding abschließen



Eine konsequente und funktionsbezogene Benennung **aller** Widgets erleichtert die Auswahl bei der Erstellung von Bindings.

Aufgabe: Wert eines BasicSlider-Widgets auf einem RadialGauge-Widget anzeigen

Ziel der Aufgabe ist es, dass innerhalb eines Contents zwei Widgets Informationen untereinander austauschen. Der Wert eines BasicSlider-Widgets soll an einem RadialGauge-Widget angezeigt werden. Beide Widgets befinden sich im "ContentMainPage".

- 1) "ContentMainPage" öffnen
- 2) RadialGauge-Widget und BasicSlider-Widget einfügen
- 3) Brease-Brease Binding für RadialGauge-Widget über "value \ Binding" erstellen:

Das **BasicSlider**-Widget stellt den Wert für das Binding zur Verfügung und ist somit die **Source** des Bindings. Das **RadialGauge**-Widget erhält den Wert des BasicSlider-Widgets und ist somit das **Target** des Bindings.



Ergebnis:

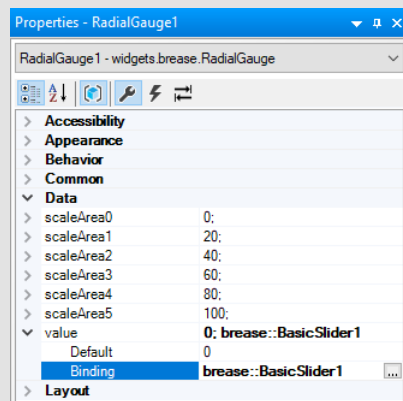


Abbildung 33: Brease-Brease Binding bei RadialGauge-Widget

Ergebnis in der Visualisierung

Auf der MainPage wird eine Bewegung des Sliders durch eine Zeigerbewegung des RadialGauge nachgestellt.

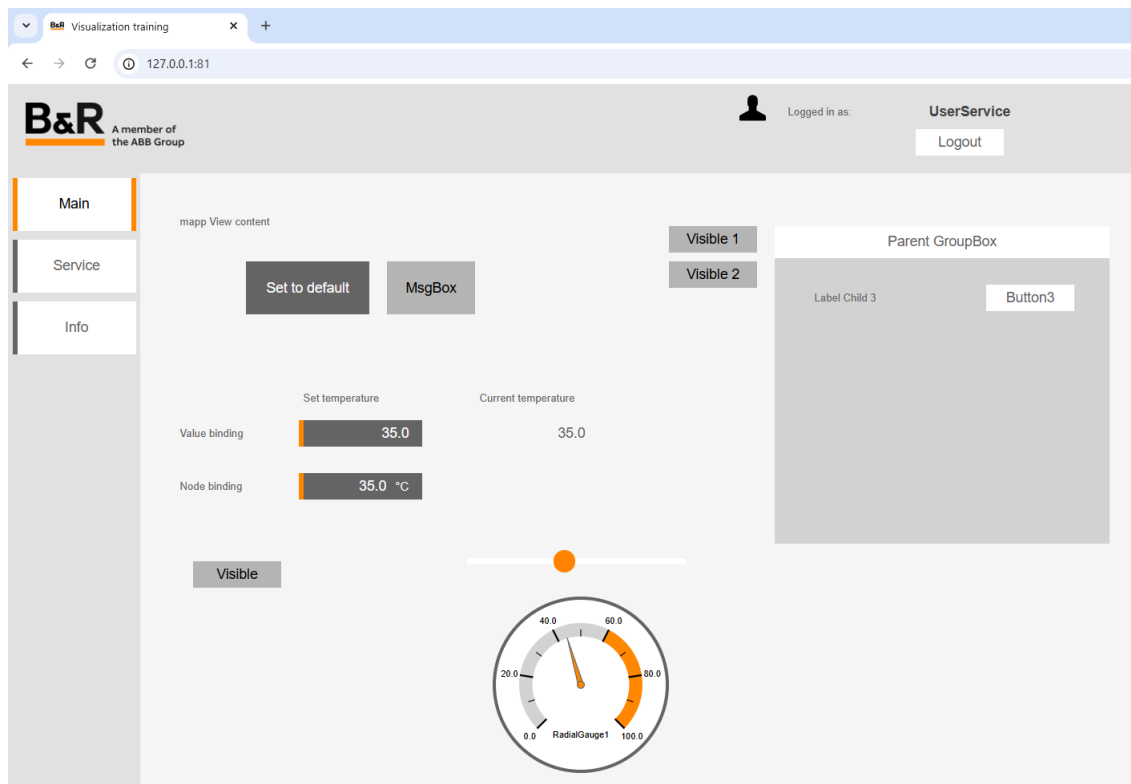


Abbildung 34: Brease-Brease Binding zwischen BasicSlider- und RadialGauge-Widget im "ContentMainPage"

5.2.2 System Variablen

System Variablen sind Session Variablen, die von mapp View fix vordefiniert sind und vom mapp View Server zur Laufzeit mit Werten befüllt werden. System Variablen können nur gelesen und nicht beschrieben werden.



[Visualization \ mapp View \ Engineering \ Variables and data \ Session variables \ System variables](#)

Aufgabe: Wert einer System Variable an einem Widget anzeigen

Ziel dieser Aufgabe ist es den Wert einer System Variable an einem Ausgabe Widget in der Visualisierung anzuzeigen. Es soll der aktuell eingeloggte Benutzer (: :SYSTEM:clientId.userId) an einem TextOutput-Widget ausgegeben werden.

- 1) TextOutput-Widget in einen beliebigen Content einfügen
- 2) Eigenschaft "value" des Widgets an eine System Variable (z.B. : :SYSTEM:clientId.userId) binden



System Variablen sind im Binding Dialog wie die Session Variablen im Tab "Variables" zu finden.

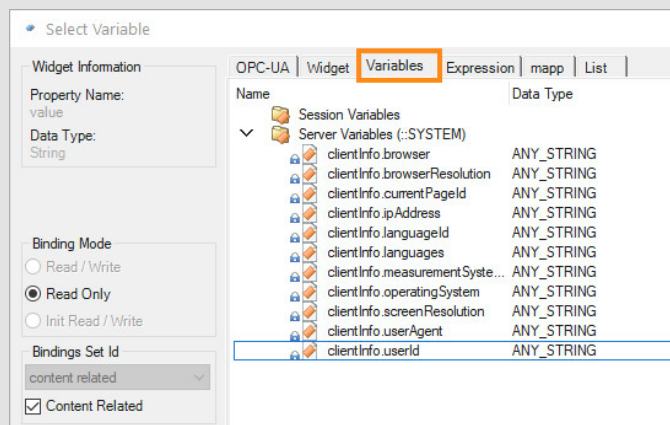


Abbildung 35: System Variablen im Binding Dialog

- 3) Anzeige in der Visualisierung testen

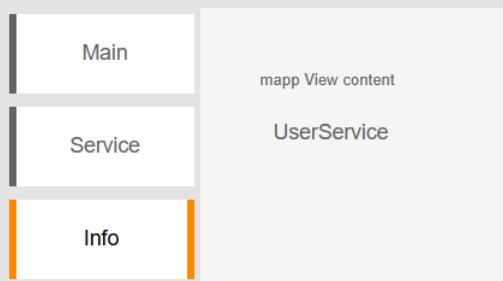


Abbildung 36: Die Information von clientId.userId wird im TextOutput-Widget angezeigt

5.2.2.1 Listen Binding

Wird ein Wert aus einer Liste von Werten benötigt, so wird ein Listen Binding verwendet.

- Der Wert des Selektors definiert, welches Element aus der Liste gewählt wird.
- Der Wert des Elements aus der Liste definiert, welcher Wert gebunden wird.

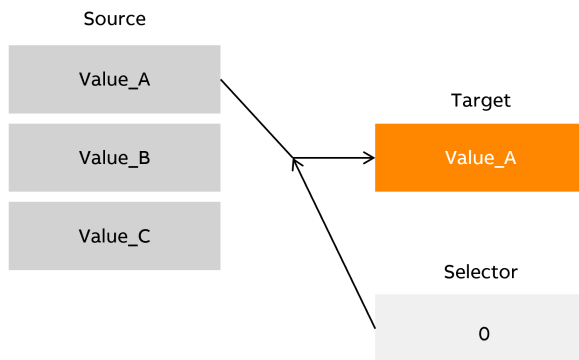


Abbildung 37: Lesen eines Wertes aus einer Liste



Listen dürfen nur als Quelle (Source) nicht aber als Ziel (Target) verwendet werden!
Sollen Werte in eine Liste **geschrieben** werden, muss der Binding Mode "oneWayToSource" (Init Read / Write) verwendet werden, da die Liste immer die <Source> sein muss.



[Visualization \ mapp View \ Engineering \ Variables and data \ Binding \ List binding](#)

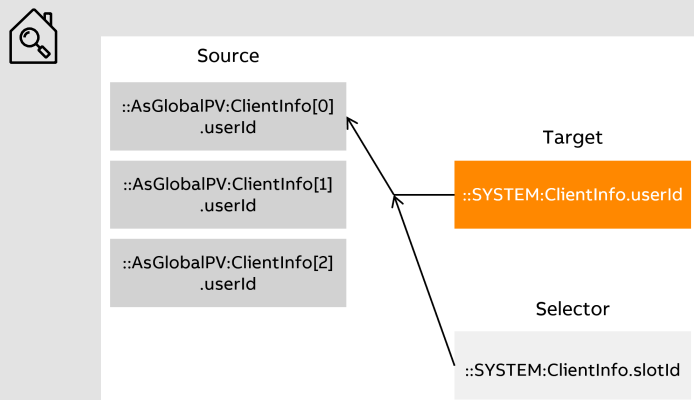


Abbildung 38: Schreiben einer Systemvariable in eine OPC UA Liste

Der Wert einer System Variable (`clientInfo.userId`) soll über den Selektor, der ebenfalls eine System Variable ist (`clientInfo.slotId`), in eine Liste geschrieben werden, die auf der Steuerung in Form eines Arrays mit entsprechenden Strukturelementen existiert (`ClientInfo[selector].userId`).

```
<Binding mode="oneWayToSource">
  <Source xsi:type="listElement">
    <Selector xsi:type="session" refId="::SYSTEM:clientInfo.slotId"
      attribute="value" />
    <be:List xsi:type="be:opcUa" attribute="value" >
      <bt:Element index="0" refId="::AsGlobalPV:ClientInfo[0].userId" />
      <bt:Element index="1" refId="::AsGlobalPV:ClientInfo[1].userId" />
      <bt:Element index="2" refId="::AsGlobalPV:ClientInfo[2].userId" />
    </be:List>
  </Source>
  <Target xsi:type="session" refId="::SYSTEM:clientInfo.userId"
    attribute="value" />
</Binding>
```

- Binding mode="oneWayToSource" zeigt an, dass in die Liste geschrieben wird
Target wird gelesen, Source wird geschrieben
 - <Source> ... </source> beschreibt die Liste aus userId Strukturelementen
 - <Selector ... refId = "::SYSTEM:clientInfo.slotId" />
die System Variable "slotId" ist der Selektor
 - <Target ... refId="::SYSTEM:clientInfo.userId" />
die Systemvariable "userId" ist das Target

Aufgabe: Listen Binding von System Variablen an OPC UA Struktur-Array

Ziel dieser Aufgabe ist es, die Client Informationen aller verbundenen Clients im Steuerungsprogramm verfügbar zu machen. Die Client Informationen sind über die System Variablen : :SYSTEM:clientInfo verfügbar und sollen über ein Listen Binding an ein Array mit entsprechenden Strukturelementen gebunden werden. Das Array mit den Strukturelementen ist Bestandteil des Steuerungsprogramms, also eine OPC UA Variable.

- 1) Das Array ClientInfo[0..2] in der OPC UA Default View aktivieren
- 2) Einfügen einer neuen Binding Datei aus der Toolbox in die Configuration View
- 3) Das Listen Binding der System Variablen an das OPC UA Struktur-Array kopieren und in die Binding Datei einfügen:



[Visualization \ mapp View \ Guides \ FAQ \ Binding applications \ Displaying client information from all clients](#)

- 4) In der watch die Daten der bestehenden Session betrachten



Ergebnis:

Die Daten der bestehenden Session sind im Eintrag ClientInfo[0] des Arrays zu sehen

Watch [Program::Main.st]	
Name	Value
ClientInfo	
ClientInfo[0]	
userId	"UserService"
isValid	TRUE
ipAddress	"127.0.0.1"
languageId	"en"
screenResolution	"1746x982"
slotId	0
ClientInfo[1]	
userId	****
isValid	FALSE
ipAddress	****
languageId	****
screenResolution	****
slotId	0
ClientInfo[2]	
userId	****
isValid	FALSE
ipAddress	****
languageId	****
screenResolution	****
slotId	0

5.3 Expressions

Expressions sind Konstrukte, die einen zuvor festgelegten logischen oder mathematischen Ausdruck auswerten und das Ergebnis der Auswertung zurückliefern. Bei der Auswertung des Ausdrucks kommen Operanden zum Einsatz, die man sich wie die Parameter einer Funktion vorstellen kann.

Das zurückgelieferte Ergebnis einer Expression kann wie eine Variable an eine Eigenschaft eines Widgets gebunden werden um beispielsweise die Sichtbarkeit oder Bedienbarkeit zu steuern. Dadurch ist es möglich die Eigenschaft eines Widgets nicht nur an einfache Datenpunkte sondern an komplexere Bedingungen zu knüpfen.



[Visualization \ mapp View \ Engineering \ Variables and data \ Expressions](#)

Bei der Verwendung von Expressions sind folgende Schritte durchzuführen:

Schritt	Datei	Ort der Datei
1. Expression Type erstellen	.expressiontype	Logical View
2. Expression Instanz erstellen	.expression	Logical View
3. Binding der Operanden	.binding	Configuration View
4. Binding des Results	.binding	Configuration View

Die genannten Schritte werden im Folgenden genauer erläutert. Die gezeigten Quellcodesnippets beziehen sich dabei auf den Anwendungsfall, dass ein Button in der Visualisierung erst dann bedienbar sein soll (Eigenschaft "enable = true"), wenn an zwei NumericInput-Widgets gültige Eingaben vorliegen.

5.3.1 Expression Type erstellen

Die Deklaration eines Expression Types erfolgt in einer .expressiontype-Datei.

Diese Datei wird über den Objektkatalog in den Ordner "Expressions" eingefügt. Darin werden alle in der Visualisierung benötigten Expression Typen deklariert.

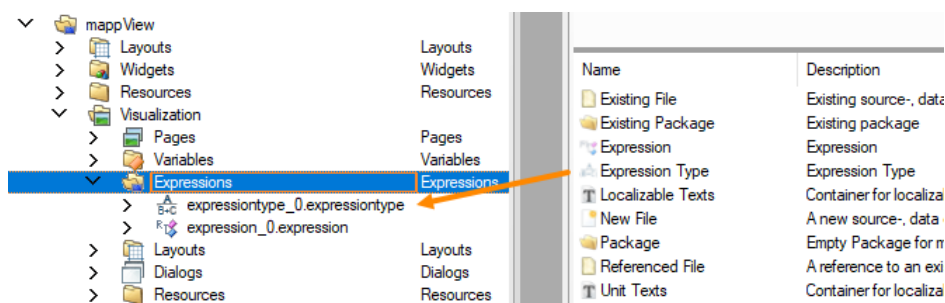


Abbildung 39: Einfügen .expressiontype Datei



[Visualization \ mapp View \ Engineering \ Variables and data \ Expressions \ Creating expressions](#)



In der .expressiontype-Datei werden – ähnlich wie bei der Programmierung von Funktionen – Typvorlagen für Expression-Instanzen erstellt.

Die Deklaration erfolgt in XML und besteht aus folgenden Elementen:

```
<ExpressionType name="InputCompleteType" datatype="BOOL">
  <Operands>
    <Operand name="NumInput1" datatype="ANY_REAL"/>
    <Operand name="NumInput2" datatype="ANY_REAL"/>
  </Operands>
  <Operation>
    NumInput1 &lt; 0 AND NumInput2 &gt; 0
  </Operation>
</ExpressionType>
```

- <ExpressionType> ist der Beginn der Expression-Deklaration
</ExpressionType> ist das Ende der Expression-Deklaration
 - name="InputCompleteType" ist der Name der Expression
 - datatype="BOOL" ist der Datentyp des Result (Rückgabewert) der Expression
- <Operands> ist der Beginn der Operanden-Deklaration
</Operands> ist das Ende der Operanden-Deklaration
Operanden kann man sich ähnlich den Parametern einer Funktion vorstellen
 - name="NumInput1" ist der Name des Operanden
 - datatype="ANY_REAL" ist der Datentyp des Operanden
- <Operation> ist der Beginn der Operation-Deklaration
</Operation> ist das Ende der Operation-Deklaration
Die Operation definiert, wie die Operanden verknüpft werden
 - Angabe des logischen oder mathematischen Ausdrucks.
Hier wird TRUE zurückgegeben, wenn NumInput1<0 und NumInput2>0 ist.



Die Symbole für kleiner (<) und größer (>) müssen in einer XML-Datei maskiert werden. Kleiner-Symbol "<" als "<" und Größer-Symbol ">" als ">"



Alle verwendbaren Datentypen von Expressions sind in Automation Help aufgelistet. Außerdem sind alle möglichen Operatoren sowie deren Bindungsstärken¹ beschrieben.

¹ Priorität von Operatoren ohne Klammerung

5.3.2 Expression Instanz erstellen

Die Deklaration einer Expression Instanz erfolgt in einer .expression-Datei.

Diese Datei wird über den Objektkatalog in den Ordner "Expressions" eingefügt.

Zum erstellten Expression Type wird eine Expression Instanz erstellt, um konkrete Werte oder Eigenschaften zu verwenden. Es können mehrere verschiedene Instanzen vom selben Type erstellt werden.

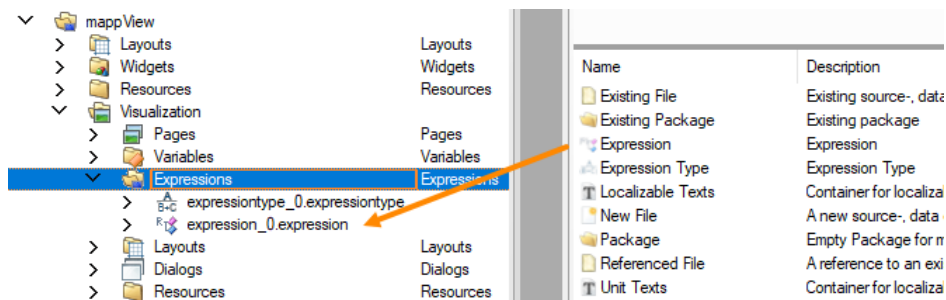


Abbildung 40: .expression Datei in Visualisierung einfügen



Die Deklaration erfolgt in XML und besteht aus folgenden Elementen:

```
<Expressions>
  <Expression id="InputCheck" xsi:type="content"
    contentRefId="ContentMainPage" type="InputCompleteType"/>
</Expressions>
```

- `<Expressions>` ist der Beginn der Expression Instanzen-Deklaration
- `</Expressions>` ist das Ende der Expression Instanzen-Deklaration
- `id="InputCheck"` ist der Namen der Expression Instanz
Mit Hilfe der ID wird die Expression Instanz referenziert
- `contentRefId="ContentMainPage"` ist der verwendete Content²
- `type="InputCompleteType"` ist der verwendete Expression Typ

² Bei contentübergreifenden Anwendungen sind entsprechend Session Variablen zu verwenden.

5.3.3 Operanden der Expression verknüpfen

Um der Expression eine tatsächliche Funktion in der Visualisierung zu geben, müssen die Operanden an OPC UA Variablen, Widgets oder Session Variablen gebunden werden.

Die Verknüpfung zwischen der Expression Instanz und der Operanden erfolgt in der .binding-Datei des jeweiligen Contents.



Das Binding erfolgt in XML und besteht aus folgenden Elementen:

```
<Binding mode="oneWay">
  <Source xsi:type="brease" widgetRefId="NumericInputNegative"
    contentRefId="ContentMainPage" attribute="value" />
  <Target xsi:type="expression" refId="InputCheck" attribute="NumInput1" />
</Binding>

<Binding mode="oneWay">
  <Source xsi:type="brease" widgetRefId="NumericInputPositive"
    contentRefId="ContentMainPage" attribute="value" />
  <Target xsi:type="expression" refId="InputCheck" attribute="NumInput2" />
</Binding>
```

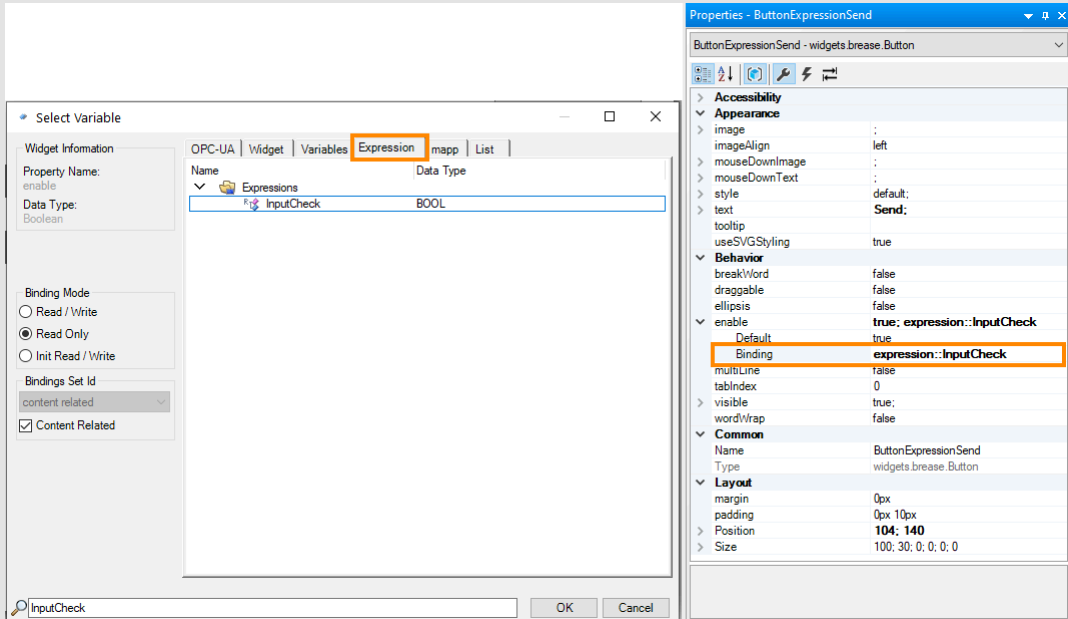
- "Binding mode=oneWay" der Wert wird von Source zu Target geschrieben
- <Source ... />
 - widgetRefId="NumericInputNegative" ist die ID des Widgets
 - contentRefId="ContentMainPage" ist der Content des Widgets
- <Target ... />
 - refId="InputCheck" ist die ID der Expression Instanz
 - attribute="NumInput1" ist der Operand der Expression

Der Wert des NumericInput-Widgets (ID = NumericInputNegative) wird auf den Operanden (name = NumInput1) geschrieben.

5.3.4 Result der Expression verknüpfen

Das Result (der Rückgabewert) der Expression wird mit einer Variable oder einem Widget verknüpft. Mündet das Result in einer **Variable**, erfolgt die Verknüpfung zwischen der Expression Instanz und des Results in der .binding-Datei des jeweiligen Contents. Mündet das Result in einem **Widget**, kann die Verknüpfung auch über den Binding Dialog erfolgen.

 Das Binding erfolgt über den Auswahldialog und benötigt folgende Schritte:



- 1) Binding für die Eigenschaft anklicken (z. B. "enable")
- 2) Im Dialog den Tab "Expression" auswählen
- 3) Gewünschte Expression Instanz auswählen (z. B. "InputCheck")

Aufgabe: Bedienbarkeit eines Buttons über eine Expression verwalten

Ziel der Aufgabe ist es die Bedienbarkeit eines Button-Widgets erst zu ermöglichen, wenn an zwei NumericInput-Widgets, die sich im selben Content befinden, gültige Eingaben gemacht wurden. Ein NumericInput-Widget soll negative, das andere positive Zahlenwerte als Eingabe erhalten, damit der Button "Send" bedienbar wird.

- 1) "ContentMainPage" öffnen
- 2) Ein GroupBox-Widget einfügen und darin zwei NumericInput-Widgets und ein Button-Widget einfügen

Widget	Common \ Name	Appearance \ text	Behavior \ min\Value	Layout \ Position
GroupBox	GroupBoxExpression	Using an expression		380; 160
NumericInput	NumericInputNegative		-100	30; 40
NumericInput	NumericInputPositive		0	30; 240
Button	ButtonExpressionSend	Send		80; 140

- 3) [5.3.1 "Expression Type erstellen" auf Seite 39](#)
- 4) [5.3.2 "Expression Instanz erstellen" auf Seite 41](#)
- 5) [5.3.3 "Operanden der Expression verknüpfen" auf Seite 42](#)
- 6) [5.3.4 "Result der Expression verknüpfen" auf Seite 43](#)

**Ergebnis:**

Solange die Eingaben an den beiden NumericInput-Widgets nicht im gültigen Bereich liegen (linker Input negativ, rechter Input positiv) ist das Button-Widget ausgegraut. Erst bei gültigen Eingaben ist die Bedienung möglich.

Abbildung 41: Gültige Eingaben

Abbildung 42: Ungültige Eingaben

5.4 Snippets

Die Deklaration eines Snippets erfolgt in einer .snippet-Datei.

Diese Datei wird über den Objektkatalog in den Ordner "Snippets" eingefügt.

Snippets sind Platzhalter, deren Inhalt aus einer anderen Datenquelle (z. B. einer OPC UA Variable) stammt. Sie werden vor allem verwendet, um variable Inhalte in Texten darzustellen.

Beispielsweise kann die aktuelle Temperatur über einen Textstring mit einem Snippet ausgegeben werden. Dafür wird im Text ein Platzhalter in Form von {@SnippetName} verwendet. Zur Laufzeit wird in der Visualisierung der Platzhalter mit dem Wert des Snippets ersetzt.

Text ID	German (de)	English (en)
1 InfoPage	Information	Info
2 LoggedIn	Angemeldet als:	Logged in as:
3 Logout	Abmelden	Logout
4 MainPage	Start	Main
5 ServicePage	Service	Service
6 SetDefault	Standard wiederherstellen	Set to default
7 SnippetText1	Temperatur {@SnippetTemp1} (dire...	Temperature {@SnippetTemp1} (direct referencing of the OPC UA variable)
8 SnippetText2	Temperatur {@SnippetTemp2} (Val...	Temperature {@SnippetTemp2} (value binding to OPC UA variable)
9 New_Text_ID		

Abbildung 43: Platzhalter für Snippet in mapp View Texten



[Visualization \ mapp View \ Engineering \ Variables and data \ Snippets](#)

[Visualization \ mapp View \ Guides \ FAQ \ Binding applications \ Live update of a value in a text](#)



Die Deklaration erfolgt in XML und besteht aus folgenden Elementen:

```
<Snippets>
  <Snippet id="SnippetTemp1" xsi:type="session" type="Numeric"
    formatItem="{#::AsGlobalPV:CurrentTemperature}" />
  <Snippet id="SnippetTemp2" xsi:type="session" type="Numeric"
    formatItem="{1|.0}" />
</Snippets>
```

- <Snippets> ist der Beginn der Snippet-Deklaration
</Snippets> ist das Ende der Snippet-Deklaration
 - id="SnippetTemp1" ist die ID des Snippets
 - xsi:type="session" ist der Geltungsbereich des Snippets
 - type="Numeric" ist der Datentyp
Möglich sind sind "Numeric", "String" oder "IndexText"
 - formatItem="" ist die typabhängige Formatierung des Wertes

Aufgabe: TextOutput-Widgets mit Snippets zur Ausgabe von CurrentTemperature erstellen

Ziel dieser Aufgabe ist es, zwei TextOutput-Widgets in die Visualisierung einzufügen, die über je ein Snippet den Wert der OPC UA Variable "CurrentTemperature" im Text anzeigen.

Eines der Snippets verwendet ein direktes Referenzieren der Variable, das andere ein Value Binding der Variable an das Snippet.

Aufgabe Teil1: Direktes Referenzieren eines Wertes an Snippet

- 1) In der Configuration View die Datei "Config.mappviewcfg" öffnen
- 2) Unter "Server configuration \ Startup User" das Feld "Resolve global resources with anonymous token " auf "TRUE" setzen
- 3) In der Logical View unter "mapp View \ Resources \ Texts" die Datei "VisualizationTexts.tmx" öffnen
- 4) Einen neuen Eintrag hinzufügen
 - Text ID: SnippetText1
 - German (de): Temperatur {@SnippetTemp1} (direkte Referenzierung der OPC UA Variable)
 - English (en): Temperature {@SnippetTemp1} (direct referencing of the OPC UA variable)
- 5) In der Logical View unter "mapp View \ Resources" auf "Snippets" klicken
- 6) Aus dem Objektkatalog "Snippet" einfügen
- 7) 5.4 "Snippets" auf Seite 45
Snippet erstellen mit ID SnippetTemp1
- 8) "ContentMainPage" öffnen
- 9) Aus dem Objektkatalog folgendes Widget einfügen und parametrieren

Widget	Common \ Name	Behavior \ breakWord	Behavior \ multiLine	Data \ value \ Default	Layout \ Postition	Layout \ Size
TextOutput	TextOutputSnippet1	true	true	\$IAT/SnippetText1	500; 60	300; 80



Eine nachträgliche Änderung des Wertes der OPC UA Variablen führt nicht zu einer Änderung des Snippets. Für eine ständige Aktualisierung des Wertes eines Snippets muss ein Werte Binding definiert werden.

Aufgabe Teil 2: Referenzieren eines Wertes mit Value Binding an Snippet

Beim direkten Referenzieren eines Wertes an ein Snippet, wird der Wert des Snippets nur dann aktualisiert, wenn der Content, der das Snippet enthält, neu geladen wird. Möchte man erreichen, dass die Aktualisierung des angezeigten Texts automatisch bei Wertänderung der OPC UA Variable erfolgt, muss das Snippet mit einem Value Binding an die OPC UA Variable erstellt werden.

- 1) In der Logical View unter "mapp View \ Resources \ Texts" die Datei "VisualizationTexts.tmx" öffnen
- 2) Einen neuen Eintrag hinzufügen
 - Text ID: SnippetText2
 - German (de): Temperatur {@SnippetTemp2} (Value Binding an OPC UA Variable)
 - English (en): Temperature {@SnippetTemp2} (value binding to OPC UA variable)
- 3) In der Logical View unter "mapp View \ Resources \ Snippets" die Datei "snippet_0" öffnen
- 4) 5.4 "Snippets" auf Seite 45
Snippet erstellen mit ID SnippetTemp2
- 5) "ContentMainPage" öffnen
- 6) Aus dem Objektkatalog folgendes Widget einfügen und parametrieren

Widget	Common \ Name	Behavior \ breakWord	Behavior \ multiLine	Data \ value \ Default	Layout \ Position	Layout \ Size
TextOutput	TextOutputSnippet2	true	true	\$IAT/SnippetText2	600; 60	300; 80

7) Nun müssen drei Bindings in der Datei "ContentMainPage.binding" erstellt werden:

- Ein Binding, das bei Änderung der OPC UA Variablen das Snippet ändert.
Source = OPC UA Variable, Target = Snippet

```
<Binding mode="oneWay">
  <Source xsi:type="opcUa" refId="::AsGlobalPV:CurrentTemperature"
    attribute="value" />
  <Target xsi:type="snippet" refId="SnippetTemp" attribute="value" />
</Binding>
```
- Ein Binding, das bei Änderung des Snippets den Text ändert.
Source = Snippet, Target = Text

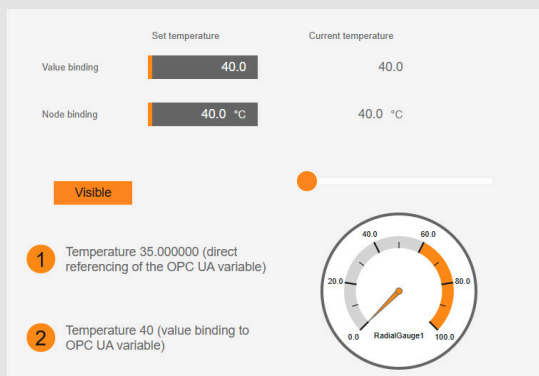
```
<Binding mode="oneWay">
  <Source xsi:type="snippet" refId="SnippetTemp" attribute="value" />
  <Target xsi:type="text" refId="IAT/SnippetText" attribute="value" />
</Binding>
```
- Ein Binding, das die Änderung des Textes in der Visualisierung sichtbar macht.
Source = Text, Target = TextOutput-Widget in "ContentMainPage"

```
<Binding mode="oneWay">
  <Source xsi:type="text" refId="IAT/SnippetText" attribute="value" />
  <Target xsi:type="brease" widgetRefId="TextOutputSnippet"
    contentRefId="ContentMainPage"
    attribute="value" />
</Binding>
```

So wird sichergestellt, dass eine Änderung des Werts der OPC UA Variable `CurrentTemperature` auch im Text laufend aktualisiert wird.



Ergebnis:



- 1) Das TextOutput-Widget verwendet ein Snippet, das den Wert der OPC UA Variable `CurrentTemperature` direkt referenziert. Es wird immer der Wert angezeigt, der beim Laden des Contents, der das TextOutput-Widget beinhaltet, aktuell war.
- 2) Das TextOutput-Widget verwendet ein Snippet, das den Wert der OPC UA Variable über Value Bindings referenziert. Über die Bindings wird der Inhalt des TextOutput-Widget immer neu generiert, wenn sich an der OPC UA Variable `CurrentTemperature` der Wert geändert hat.

Abbildung 44: Snippet Binding Arten im Vergleich

6 Multi Client Anwendungen

Unterschiedliche Benutzer benötigen individualisierte Benutzerinhalte. mapp View ermöglicht rollenspezifische Anzeigen, ohne diese in der Maschinenapplikation programmieren zu müssen.

Die individualisierten Visualisierungsseiten können zeitgleich auf unterschiedlichen Ausgabegeräten angezeigt werden.



Es wird nicht empfohlen, die Maschinenvisualisierung für mobile Endgeräte auszuliefern, da die Bedienung über z. B. ein Tablet meist einer spezifischen Rolle wie z. B. einem Servicetechniker entspricht. mapp View bietet die Möglichkeit, eigenständige Visualisierungen (.vis) mit den für den Einsatzfall spezifischen Pages zu erstellen, wobei die Wiederverwendung bestehender Contents in den Pages möglich ist.

In dieser Aufgabe wird gezeigt, wie sich mehrere Clients auf eine Visualisierung verbinden können. In der mapp View Konfiguration wird festgelegt, wie viele Clients sich gleichzeitig auf den mapp View Server verbinden dürfen. Dabei ist es unerheblich, welche Visualisierung an den Client ausgeliefert wird.

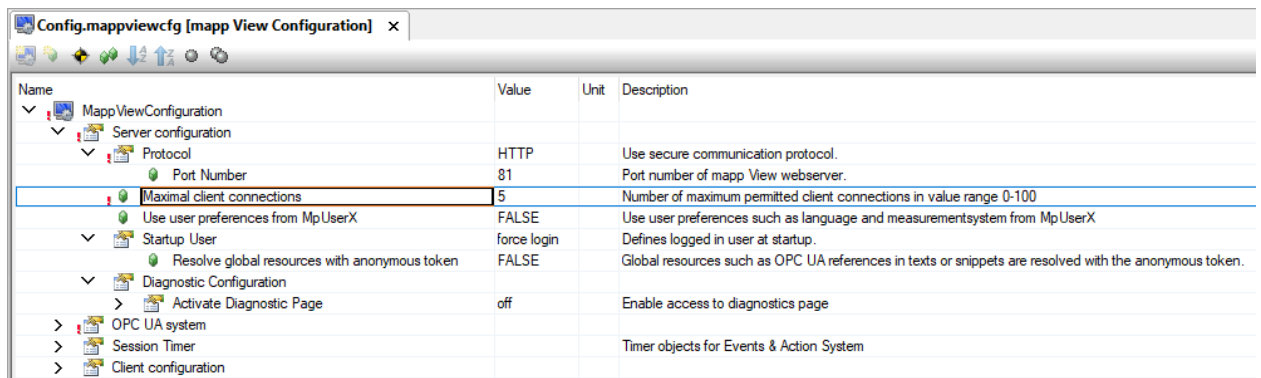


Abbildung 45: Erlauben von 5 Client Verbindungen



Im Beispiel sollte die Anzahl der maximal erlaubten Client Verbindungen mit der Länge des OPC UA Arrays `ClientInfo` übereinstimmen, da es beim im vorigen Kapitel konfigurierten Listenbinding zu einem Zugriff auf nicht erlaubte Indexwerte kommen kann.

In den bisherigen Aufgaben wurde die Visualisierung in einem Browserfenster angezeigt.

Wird der Browser für den Test einer zweiten Client Verbindung ein zweites Mal mit der gleichen URL geöffnet, wird dies vom mapp View Server abgelehnt, da von diesem Browsertyp die Session bereits "belegt" ist. Dabei ist es unerheblich, ob es zwei Browserfenster oder Browsertabs gestartet werden.

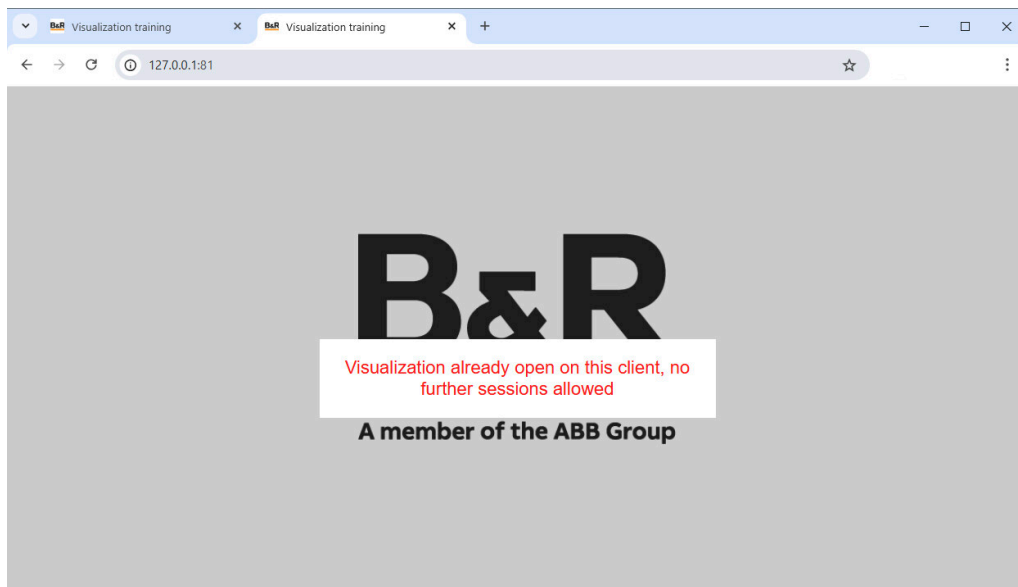


Abbildung 46: Fehlermeldung wenn sich der Browser auf die gleiche Session verbindet

Zum Testen der Visualisierung mit zwei Clients bieten einige Browser die Möglichkeit, eine zweite Instanz im sogenannten "Inkognito Modus" zu öffnen. In diesem Fall wird eine neue Verbindung (Session) auf den mapp View Server erzeugt.

Damit können zwei Client Instanzen in je einem eigenen Browserfenster angezeigt und individuell bedient werden. Auf jedem Client kann ein unterschiedlicher Benutzer eingeloggt sein. Auch die Sprache und das Maßsystem kann auf jedem Client individuell ausgewählt werden.

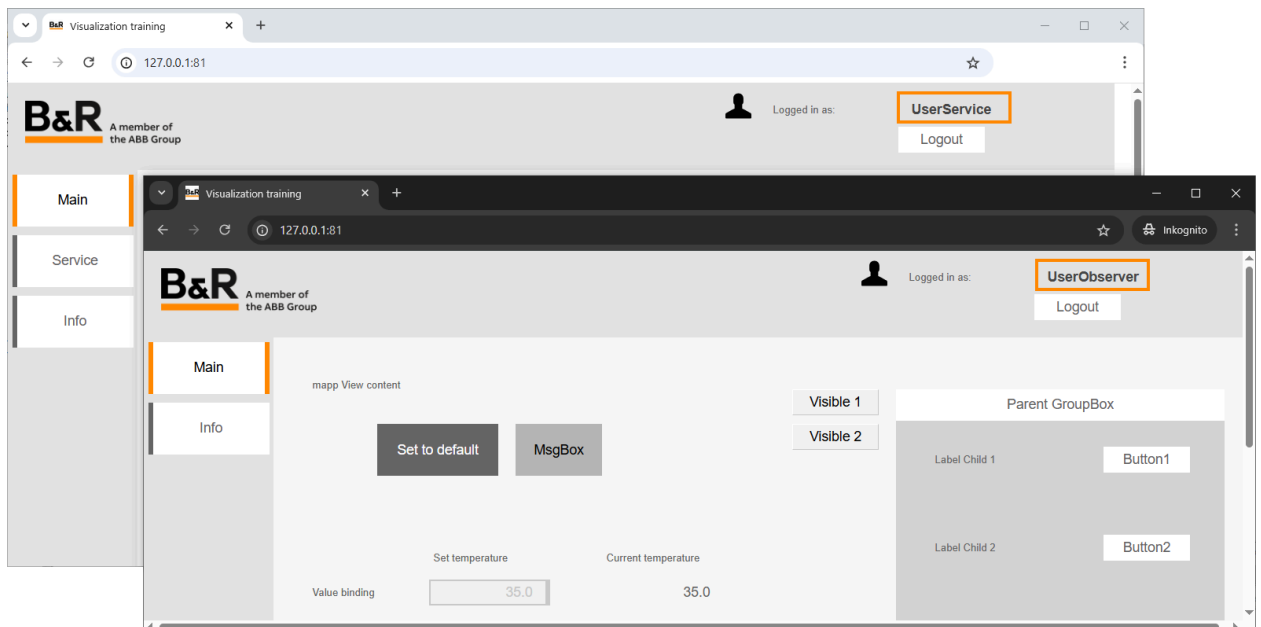


Abbildung 47: Anzeige der Visualisierung mit zwei Client Instanzen

Aufgabe: Verbindung mehrerer Clients erlauben

Ziel dieser Aufgabe ist es mehrere Verbindungen zum mapp View Server zu erlauben und anschließend zu testen.

- 1) Anzahl der maximal erlaubten Client Verbindungen in der mapp View Konfigurationsdatei (Config View) auf 5 ändern (ClientInfo Array hat fünf Elemente)
- 2) Zweites Browserfenster im Inkognito Modus öffnen und als beliebiger Nutzer einloggen
- 3) Verbindungsdetails beider Sessions in der Watch betrachten

7 Dynamisch veränderbare Grafiken

In vielen Visualisierungen gibt es die Anforderungen Prozessbilder und Grafiken in Abhängigkeit des Zustands und des Status der Maschinenapplikation zu verändern. Für diesen Zweck bietet mapp View das Paper-Widget an. Es stellt einen Grafikbereich zur Verfügung, über welchen Elemente von SVG³ Grafiken angezeigt und verändert werden können.

SVG Grafik erstellen

Details zur Erstellung von SVG Grafiken sind auf w3schools.com zu erfahren.
(siehe www.w3schools.com/graphics/svg_intro.asp)



[Visualization \ mapp View \ Widgets \ Drawing \ Paper](#)

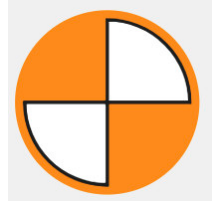


Abbildung 48: Beispiel für eine rotierende Scheibe

Die SVG Datei "MotorShaft.svg" kann zur Kontrolle in einem externen Texteditor geöffnet werden.

Mögliche Transformationen

Die unterstützten Transformationen werden kurz in der Automation Help erklärt. Transformationen, also die Bewegungen der SVG Grafik können in Form einer Stringvariable an das Paper-Widget gebunden werden.



[Visualization \ mapp View \ Widgets \ Drawing \ Paper \ Concept](#)



Durch Umschalten einer Prozessvariable vom Typ String, wird im Widget jeweils eine Transformation aktiviert. So findet die Animation in der Grafik statt.

Aufgabe: Paper-Widget projektieren

Ziel dieser Aufgabe ist es, das Paper-Widget zur Animation einer Grafik in einer mapp View Visualisierung zu verwenden. Verwendet wird eine SVG Grafik, welche eine rotierende Scheibe darstellt. In der SVG-Datei gibt es ein Element, über welches mit Hilfe einer Transformation in Rotation versetzt wird.

- 1) In der OPC UA Default View die Variable "transformation" (enthält einen String mit Transformationsanweisungen) aktivieren
- 2) "ContentInfoPage" öffnen
- 3) Paper-Widget einfügen und Eigenschaften konfigurieren

Common \ Name	Data \ svgFilePath \ Default	Data \ transform \ Binding	Layout \ Position	Layout \ Size
PaperMotorShaft	Media/Images/MotorShaft.svg	opcUa()::Program:transformation	40; 450	640;380

³ Scalable Vector Graphics ist die vom World Wide Web Consortium empfohlene Spezifikation zur Beschreibung zweidimensionaler Vektorgrafiken. SVG, das auf XML basiert, wurde erstmals im September 2001 veröffentlicht. (Quelle: de.wikipedia.org)



Ergebnis:

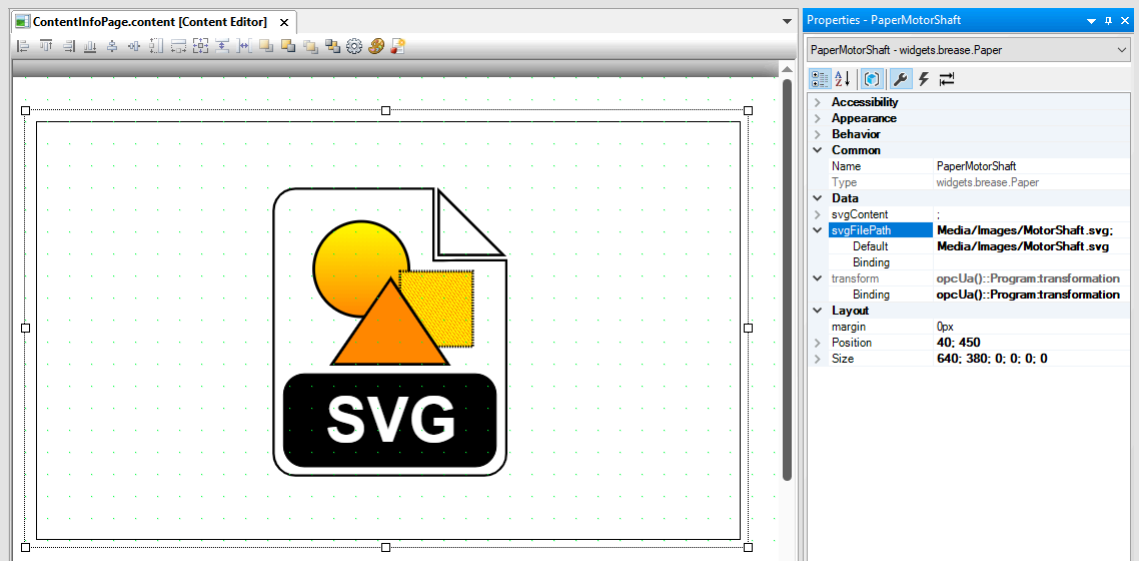


Abbildung 49: fertig projektiertes Paper-Widget



Im Steuerungsprogram existiert ein Array `UseCaseAnimations[0..6]` mit 7 Stringzeilen, die über `TransformationIndex` durchgeschaltet werden können.

Durch Umschalten der Prozessvariable `TransformationIndex` in der watch wird jeweils eine anderer Transformationsstring aus dem Array in die an das Widget gebundene Stringvariable `transformation` kopiert.

8 Zusammenfassung

Durch das selbstständige Erarbeiten der Aufgabenstellungen vertieften und festigten Sie Ihre mapp View Basiskenntnisse. Sie lernten, das Event & Action System sowie das Benutzer-Rollen-System für verschiedene Anwendungsfälle zu nutzen und Widgets zu dynamisieren. Zudem erhielten Sie einen Überblick über den Einsatz von Visualisierungen auf unterschiedlichen Endgeräten und die optische Darstellung von Maschinenprozessen.

Automation Academy

Ihr Wissensvorsprung

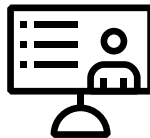
Die Automation Academy ist für die **zielgerichtete Weiterbildung** unserer Kunden und eigenen Mitarbeiter verantwortlich. Erweitern Sie Ihre Kompetenzen auf dem Gebiet der Automatisierungstechnik und lernen Sie selbstständig **effiziente Automatisierungslösungen** mit B&R-Systemen zu realisieren.

Entscheiden Sie selbst, mit welchem **Lernkonzept** Sie lernen möchten!



Präsenztraining

Ein erfahrener Trainer führt Sie durch das Lernprogramm. Vor Ort am gewünschten B&R-Standort. Lernen individuell oder in kleinen Lerngruppen.



Virtuelles Klassenzimmer

Ein ortsunabhängiges Fernstudium ergänzt das Lernangebot. Ein Online-Tutor begleitet Sie virtuell. Der Schwerpunkt liegt auf dem Selbststudium.



Onlinekurse

Sie erarbeiten sich Ihr Wissen selbstständig und bestimmen Tempo und Inhalte selbst. Onlinekurse sind jederzeit verfügbar und unabhängig von Lernzeit und Lernort.

Kontakt

Sie haben ein aktuelles Weiterbildungsanliegen? Sie interessieren sich für die Angebote der B&R Automation Academy? Bei uns sind Sie genau richtig!

Hier erhalten Sie weitere Informationen:

<https://www.br-automation.com/de/academy/>

Viel Spaß bei Ihrem nächsten Training!



AutomationAcademy





B&R
Industrial Automation GmbH
A member of the ABB Group
B&R Straße 1
5142 Eggelsberg, Austria
office@br-automation.com

t +43 7748 6586-0
f +43 7748 6586-26

br-automation.com

